arXiv:2509.17876v3 [quant-ph] 7 Oct 2026

Quantum Portfolio Optimization: An Extensive Benchmark

Eric Stopfer[1,*] and Friedrich Wagner[1]

[1]Fraunhofer Institute for Integrated Circuits, Nürnberg
[*]eric.stopfer@iis.fraunhofer.de

October 2026

Abstract

Recently, several researchers proposed portfolio optimization as a potential use case for quantum optimization. However, the literature is lacking an extensive benchmark quantifying the potential of quantum computers for portfolio optimization. In this work, we contribute to closing this gap. We provide a computational study, comparing quantum approaches against state-of-the-art classical methods on a meaningful, real-world instance set. In particular, we compare quantum annealing and the quantum approximate optimization algorithm against classical mixed-integer programming, simulated annealing, steepest descent local search, tabu search and a problem-tailored heuristic. We consider a variance-minimizing variant of portfolio optimization which we show to be more difficult to solve for classical optimizers than return-maximizing or multi-objective formulations. Our benchmark data set comprises 270 instances with up to 1,000 assets from actual stock data. Due to hardware limitations, quantum methods could only be tested for instances with at most 30 assets. The results show that all instances can be solved to proven optimality by mixed-integer programming in the order of seconds. Moreover, the problem-tailored heuristic consistently outperforms quantum approaches in terms of solution quality for fixed runtime. Thus, we conclude that there is only very limited room for a potential quantum advantage for the considered variant of portfolio optimization.

1 Introduction

Combinatorial optimization problems play a central role in various fields such as logistics and finance [1]. In practice, these problems are often tackled by mathematical optimization solvers, which can solve large-scale mixed-integer and quadratic optimization problems, including many NP-hard instances, to proven optimality [2, 3]. Nevertheless, some practically relevant problems remain intractable for state-of-the-art classical methods [4]. This fact has motivated researchers to develop quantum algorithms for such problems [5]. However, the limitations of existing quantum computers prohibit the implementation of exact quantum algorithms for optimization problems [6]. As a result, researchers focus on heuristic quantum algorithms, which require significantly less resources than exact approaches [7]. Well-known examples of such quantum heuristics are quantum annealing [8, 9] and the quantum approximate optimization algorithm (QAOA) [10, 11]. Recently, several works proposed portfolio optimization as a suitable problem class for achieving quantum advantage [12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22]. In this work, we provide experimental evidence suggesting that even large instances of portfolio optimization can be solved to proven optimality by modern classical solvers in the order of seconds. Moreover, our problem-tailored classical heuristic outperforms quantum approaches in terms of solution quality for fixed runtime. Our results thus set the bar high for a practical quantum advantage for the considered variant of portfolio optimization.

Related work. Various variants of the portfolio optimization problem exist in literature. The authors of [15, 16, 18, 19] consider a variant that can be described as *choose-asset-or-not*. Therein, as the name suggests, the task is to decide for each asset if we add it to the portfolio or not, equally weighted. In this work, we consider an extended variant where the goal is to find optimal *weights* of assets in the portfolio, that is, we assign a fraction of the invested money to each asset. This variant can be further subdivided with respect to the considered objective. Some researchers focus on maximizing return with limited return variance [13, 17]. Others aim to minimize the variance with a fixed return [18, 23]. Several authors combine both objectives by maximizing a weighted sum of return and variance [15, 19, 24].

There also exist approaches in literature which include further practical aspects of stock trading into the model, for example, short-selling-options, risk-free investments and costs that are associated with asset acquisitions [12, 20, 24]. The authors of [12] consider a data test set consisting of 10, 50, 200 and 400 assets of the S&P500 stock market which is similar in structure but less granular than the data used in this work.
Classical solution approaches to the portfolio optimization problem include both heuristics [24, 25, 26, 27] and exact methods [28, 29, 30]. Notably, for fixed runtime, the heuristic developed in [30] finds solutions of better quality for instances of the choose-asset-or-not problem than the commercial classical solver CPLEX [31].
Recently, researchers proposed quantum approaches to the portfolio problem, including QAOA [19, 20], Variational Quantum Eigensolver (VQE) [22] and quantum annealing [13, 16, 17, 18, 21]. The authors of [13], [16] and [17] conclude that quantum annealing may outperform classical methods for portfolio optimization. However, their studies employ a closed-source hybrid quantum-classical solver, which makes it impossible to divide classical from quantum contributions to the solution. On the contrary, in this work, we stick to methods with known classical and quantum components. Regarding QAOA, the authors of [20] report near-optimal results for small problem instances of 8 assets with an idealized simulator of a gate-based quantum computer. Ref. [19] provides guidelines on the selection of penalty and circuit parameters as well as on the classical optimizer. In [18, 21], the authors use reverse quantum annealing to improve upon conventional quantum annealing. Ref. [22] studies the influence of penalty coefficients in VQE as well as the effects of different quantum devices. Other approaches cover problem sparsification [26], problem decomposition [15], warm-starts [32] and quantum circuit cutting techniques [33]. For a comprehensive review on solution approaches to the portfolio optimization, we refer the interested reader to [34].

Our contribution. In this work, we provide a computational study, including 270 problem instances with up to $1,000$ assets. The instances are generated from real-world stock data providing instances with realistic return and covariance. However, we note that we do not make any tests based on actual investment performance. Moreover, we find optimal portfolio weights instead of only deciding whether to select an asset or not. Thus, our approach resembles practical scenarios. We compare QAOA, quantum annealing, two exact classical solvers, three meta-heuristics and one problem-specific heuristic. Our contribution is twofold. First, the large instance set allows us to draw conclusions that are consistent across different instances and instance sizes. Second, applying standard implementations of state-of-the-art quantum and classical solution methods, we provide a comprehensive and fair benchmark. In particular, our methods have well-defined quantum and classical parts such that we can precisely divide quantum from classical contributions.
The remainder of this work is organized as follows. In section 2, we define three variants of portfolio optimization which we consider in this work. Section 3 studies the classical hardness of these variants to identify the one with the largest potential for quantum advantage. In section 4, we develop a quadratic unconstrained binary optimization (QUBO) model for the most difficult portfolio optimization variant. Section 5 introduces the solution methods we compare in our benchmark. In Section 6, we report on the results of our benchmark study. Finally, in section 7, we summarize our findings, draw a conclusion and state open questions.

2 The Portfolio Optimization Problem

In this section, we introduce three different variants of the portfolio optimization problem, which we consider in our computational study. As originally formulated by Markowitz in [35], the portfolio optimization problem asks for a selection of an asset portfolio which maximizes the expected portfolio return while minimizing the variance of the return..
More formally, given a set of n assets, the task is to find optimal *asset weights* $\omega_i \in [0,1]$, $i \in \{1,\dots,n\}$. The asset weight ω_i defines the share of the invested money allocated to the i-th asset. Accordingly, it holds

$$\sum_{i=1}^{n} \omega_i = 1 \ .$$

We impose upper bounds $u_i \in (0,1]$ on the asset weights,

$$\omega_i \leq u_i \quad \forall i \in 1,\dots,n.$$

Each asset i has an *expected return* $r_i \in \mathbb{R}$. Moreover, for each pair of assets (i, j), we are given a *return covariance* $\sigma_{ij} \in \mathbb{R}$.
The original problem formulation by Markowitz [35] includes the objectives of maximizing the expected portfolio return

$$\mu(\omega) \coloneqq \sum_{i=1}^{n} \omega_i r_i$$

and minimizing the portfolio variance

$$\sigma^2(\omega) \coloneqq \sum_{i=1}^{n} \sum_{j=1}^{n} \omega_i \omega_j \sigma_{ij} \ .$$

In general, the two objectives of maximizing return and minimizing variance cannot be fulfilled at the same time. Thus, several problem variants exist, which we summarize in the following. In the first variant, we aim at maximizing the return with an upper bound $\nu > 0$ on the variance. This can be formulated by the quadratic program

$$\begin{aligned} \textbf{MaxRet}: \quad \max_{\omega} \quad & \mu(\omega) \\ \text{s.t.} \quad & \sigma^2(\omega) \leq \nu \\ & \sum_{i=1}^{n} \omega_i = 1 \\ & 0 \leq \omega_i \leq u_i \quad \forall i \in \{1, \dots, n\} \ . \end{aligned}$$

The second variant minimizes the variance while ensuring a minimum return $\epsilon \in \mathbb{R}$. This is formalized by the quadratic program

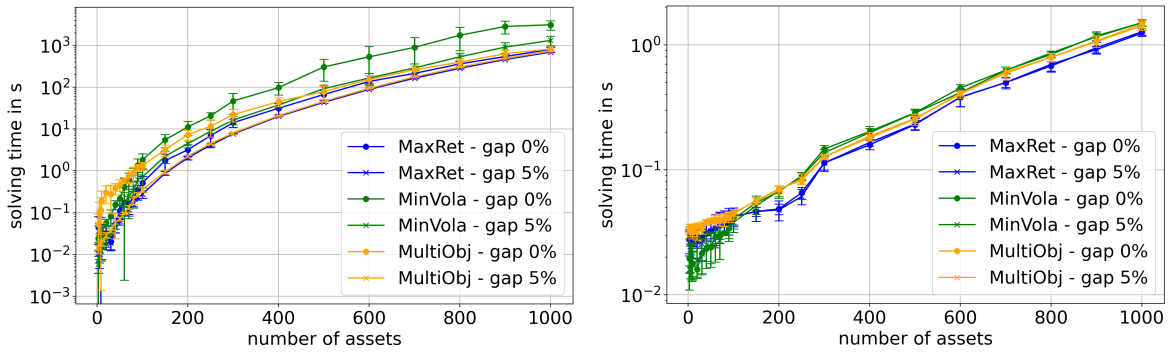
$$\begin{aligned} \textbf{MinVola}: \quad \min_{\omega} \quad & \sigma^2(\omega) \\ \text{s.t.} \quad & \mu(\omega) \geq \epsilon & (1) \\ & \sum_{i=1}^{n} \omega_i = 1 & (2) \\ & 0 \leq \omega_i \leq u_i \quad \forall i \in \{1, \dots, n\} \ . \end{aligned}$$

The name of the QP reflects its objective of minimizing portfolio volatility. Finally, the third variant combines both objectives by a fixed factor $\lambda > 0$, which depends on the risk aversion of the investor. This gives rise to the quadratic program

$$\begin{aligned} \textbf{MultiObj}: \quad \min_{\omega} \quad & \sigma^2(\omega) - \lambda \mu(\omega) \\ \text{s.t.} \quad & \sum_{i=1}^{n} \omega_i = 1 \\ & 0 \leq \omega_i \leq u_i \quad \forall i \in \{1, \dots, n\} \ . \end{aligned}$$

The mathematical programs MaxRet, MinVola and MultiObj belong to the class of convex quadratically constrained quadratic programs, which can be solved in polynomial time by interior point methods [36, 37]. The convexity is due to the fact that $(\sigma_{ij}) \in \mathbb{R}^{n \times n}$ is a sample covariance and thus positive semidefinite by definition (6) in Appendix A. Although being efficiently solvable in theory, portfolio optimization is often tackled via heuristics in practice.

Test set generation. In order to generate a realistic set of instances, we consider the daily prices of 1,978 assets in the Nasdaq stock exchange [38] during the years 2020 to 2023, available in [39] and accessible with [40]. All those assets have no missing price data during this timeframe. From the daily prices, we calculate the expected returns and the return covariances. For details on the calculation of returns and covariances from price data, we refer to Appendix A. To generate an instance with a given number n of assets, we randomly draw n assets from the data base.
We generate 10 instances for each problem variant MaxRet, MinVola and MultiObj for each size $n \in \{3, 5, 7, 8, 10, 15, 20, 25, 30, 40, 50, 60, 70, 80, 90, 100, 150, 200, 250, 300, 400, 500, 600, 700, 800, 900, 1000\}$. We still



(a) SCIP (b) Gurobi

Figure 1: Average solver runtime for different problem variants. We compare SCIP (a) and Gurobi (b) with relative optimality gaps of 0 % and 5 %. Data points are averages over 10 instances and error bars show the empirical standard deviation.

need to define values for the maximum portfolio variance ν in MaxRet, the minimum portfolio return ϵ in MinVola and the risk aversion λ in MultiObj. As we strive towards an above-average performing portfolio, in Model MaxRet, we choose the maximum portfolio variance ν as the 30 % quantile of variances of randomly generated portfolios. Similarly, in Model MinVola, we choose the minimum portfolio return ϵ as the 70 % quantile of all asset returns. Moreover, in order to account for the different magnitudes of μ and σ^2, we set the risk aversion factor λ of Model MultiObj to the average of fraction $\frac{\sigma^2}{\mu}$ of randomly generated portfolios. We remark that, in general, the selection of the parameters ϵ, ν, λ is subject to the investor's perspective. Finally, we set the asset limits to $u_i = \max\left(\frac{1}{10}, \frac{3}{n}\right)$ for $i \in \{1, ..., n\}$. The asset limits thus form a monotonically decreasing series, which for 5 assets has a value of 60%, for 10 assets 30%, for 20 assets 15% and stays at 10% from $n = 30$ onward. By this formula, we try to emulate real-world fund compositions, which can be monitored in [39].
In principle, some generated test instances of MinVola and MaxRet could be infeasible. However, our computational experiments revealed that all considered test instances are indeed feasible.

3 Classical Solution Time for Different Variants

In this section, we study the classical computational difficulty of the three problem variants MaxRet, MinVola and MultiObj. Our goal is to identify the hardest variant for classical solvers which thus bears the largest potential for a quantum advantage. To this end, we solve all instances of the test set described in Section 2 with classical mathematical optimization solvers to proven optimality and compare the overall runtime. For each instance, we solve the three problem variants with the open-source solver SCIP 9.2 [41, 42] and the commercial solver Gurobi 11.0.3 [43]. Moreover, we impose a time limit of 3,600 s and relative optimality gaps of 0 % and 5 %. Here, the relative optimality gap, short gap, is defined as

$$g := \frac{|b - c|}{|c|}$$

where b is the best bound on the objective and c is the objective value of the best available solution.
The average solving times are visualized in Figure 1.
First, we observe that Gurobi solves most of the problems in less than one second and thus is considerably faster than the open-source solver SCIP, with a speed-up factor of over 1,000 for large problem instances with $n = 1,000$ assets. The speed-up can partly be explained by the fact that Gurobi detects the convexity of the problem and consequently solves it using the Barrier algorithm, a polynomial-time interior-point method. In contrast, as observed in the solver logs, SCIP relies on outer approximation with spatial branching, which is a method for general, non-convex QPs and has exponential worst case running time.
Second, the influence of the duality gap on the solving time is larger for SCIP than for Gurobi.
Third, the Model MinVola takes the longest time to be solved in each of the four solver-gap configurations. Thus, MinVola is the most promising variant for a potential quantum speedup. Consequently, in the following we focus on the MinVola variant that minimizes variance while still maintaining a certain portfolio return level.

4 QUBO Transformation

Existing implementations of both quantum annealing and QAOA require the input problem to be modeled as QUBO, that is, problems of the form

$$\min_{x\in\{0,1\}^n} x^t Q x$$

where $n \in \mathbb{N}$ and $Q \in \mathbb{R}^{n\times n}$ is a real-valued matrix. In this section, we derive a QUBO model of the MinVola variant. To this end, we define the penalty-based objective function

$$f(\omega) \coloneqq \sigma^2(\omega) + \phi\left(\mu(\omega) - \hat{\epsilon}\right)^2 + \psi\left(\sum_{i=1}^{n}\omega_i - 1\right)^2, \tag{3}$$

where $\omega \in [0,1]^n$ are the asset weights and $\phi, \psi \in \mathbb{R}_{\geq 0}$ are penalty factors that penalize violations of the return constraint (1) and the normalization constraint (2), respectively. Here, we replace the "$\geq$" in the return constraint (1) by "=". According to Markowitz portfolio theory the variance-minimal portfolio for $\mu(\omega) \geq \xi$ has exactly return $\mu(\omega) = \xi$ for a sufficiently large ξ. We elaborate on this in appendix B. Also, we select the return target as $\hat{\epsilon} = \epsilon + 1\%$, as sampling-based methods return a distribution of portfolios whose returns scatter around the penalised target. Since a feasibility analysis accepts any solution satisfying the original inequality $\mu(\omega) \geq \epsilon$, shifting the penalised target slightly upwards increases the number of samples that are feasible for the original problem. The same target $\hat{\epsilon}$ is used for all QUBO-based methods. Thus, the comparison between the solving methods in the following chapter remains unaffected. From a theoretical point of view, finding optimal penalty factors is itself an NP-hard problem [27]. Consequently, we employ a heuristic penalty approach in this work, which we detail in section 6.
In order to derive a valid QUBO model, the continuous variables $\omega_i \in [0,1]$ in (3) have to be transformed to binaries. As proposed in [13, 17], we write the portfolio weights $\omega_i \in [0,1]$ as a linear combination of binary variables $\omega_{ij} \in \{0,1\}$, $j \in \{1,\dots,d+1\}$, via

$$\omega_i = u_i \cdot \left(\frac{1}{2^d}\omega_{i,d+1} + \sum_{j=1}^{d}\frac{1}{2^j}\omega_{i,j}\right) \quad \forall i \in \{1,...,n\}. \tag{4}$$

Here, $d \in \mathbb{N}$ controls the precision of discretization. An advantage of this discretization is the implicit incorporation of the upper bounds u_i on the asset weights. Inserting (4) in (3) yields the QUBO model for MinVola.

5 Methods

In this section, we introduce the solution methods that we compare in our benchmark. Furthermore, we explain how associated parameters are chosen. For all methods, we leave parameters to default values if not stated otherwise.

Exact mathematical optimization solver. In this benchmark, we chose Gurobi [43] as the classical solver that provably solves the problem to optimality. In order to ensure fairness when comparing the other methods to the exact solver, we solve the discretized version of the formulation MinVola.

Quantum annealing. Quantum annealing is a heuristic algorithm that runs on analog quantum computers [44, 45]. The concept of quantum annealing is based on the adiabatic theorem of quantum mechanics [46]. Quantum annealing prepares a quantum mechanical system in the ground state of some initial Hamiltonian H_0. Then, it evolves the system according to

$$H(t) = A(t)H_0 + B(t)H_1\,.$$

Here, H_1 is the problem Hamiltonian whose ground state encodes the solution to some optimization problem. $A : [0,\tau] \to [0,1]$ is chosen such that $A(0) = 1$ and $A(\tau) = 0$, where $\tau > 0$ is called *annealing time*. Similarly, $B : [0,\tau] \to [0,1]$ is chosen such that $B(0) = 0$ and $B(\tau) = 1$. Now, the adiabatic theorem states that the system is in the ground state of H_1 at $t = \tau$ for $\tau \gg 1$ if the energy gap between the ground state and the first excited state is non-zero for all $t \in [0,\tau]$ and if $\partial_t H(t)$ is finite for all $t \in [0,\tau]$.

In our computational experiments between the months of June and September of 2025, we use a D-Wave Advantage 2 processor which has more than 4,400 physical qubits, arranged in the Zephyr topology [47]. However, the limited device connectivity requires an embedding of each QUBO variable into possibly multiple physical qubits [48, 49]. This embedding overhead increases with the density of the QUBO problem. All considered QUBO instances have a density of 100%. For embedding, we therefore use the `DWaveCliqueSampler` [50] from the python package *dwave-system* [51], which employs pre-defined embeddings for fully connected QUBO problems and is thus more suited for dense problems than constructive embedding algorithms like the default `MinorMiner` [52]. Also, we test annealing times of 1 µs, 5 µs, 20 µs and 50 µs. The *annealing schedule*, which defines $A(t)$ and $B(t)$, is kept as default. However, we vary the *chain strength* parameter which can prevent the breaking of a chain of connected qubits representing a single QUBO variable. We explicitly decided against more sophisticated quantum annealing concepts like warm-starts [32] or special annealing schedules [18, 21, 53] because our goal is to test the quality of solutions returned by the basic provided infrastructure without focusing on presolving and fine-tuning the methodology and the associated parameters.

QAOA. The quantum approximate optimization algorithm is a quantum-classical hybrid algorithm, originally proposed by Farhi et al. [10]. Like quantum annealing, QAOA is conceptually based on the adiabatic theorem of quantum mechanics [46]. However, in contrast to quantum annealing, QAOA is designed for gate-based, universal quantum computers. QAOA depends on real-valued parameters $\gamma = (\gamma_1, ..., \gamma_p)$ and $\beta = (\beta_1, ..., \beta_p)$. The hyper-parameter $p \in \mathbb{N}$ controls the complexity of the algorithm. QAOA starts with the uniform superposition quantum state $|+\rangle^N$ and evolves it according to

$$|\psi(\beta,\gamma)\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \ ... \ e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^N . \tag{5}$$

Here H_C is the so-called *problem Hamiltonian*, defined by

$$H_C |x\rangle = C(x) |x\rangle \quad \forall x \in \{0,1\}^N$$

where $C(x)$ is the QUBO cost function. H_M is called the *mixing Hamiltonian* and is defined by

$$H_M = \sum_{i=1}^{N} X_i$$

where X_i is the Pauli X gate acting on the i-th qubit. For given parameters β and γ, solutions to the QUBO problem are sampled from the quantum circuit implementing (5). In this work, we employ three different methods to compute the parameters β and γ. First, for $p = 1$, we perform a grid search by analytically calculating the expectation value of the resulting QAOA circuit via the formula developed in [54]. We choose the parameter values that result in the minimum expectation value. Second, we use the linear ramp QAOA (LR-QAOA) protocol introduced in [55]. Accordingly, we set $\beta_i = (1 - \frac{i}{p})\Delta_\beta$ and $\gamma_i = (\frac{i+1}{p})\Delta_\gamma$ for $i \in \{1, \dots, p\}$. In our experiments, we choose $\Delta_\beta = 0.3$ and $\Delta_\gamma = 0.6$ as proposed in [55]. Finally, we optimize the parameters with the local optimizer COBYLA [56, 57] using a noiseless quantum simulator [58, 59]. Here, we initialize the parameters with the LR-QAOA-formula.
After selecting parameters, we execute QAOA on the gate-based quantum computer *ibm_strasbourg* based on a 127 qubit *Eagle* processor [60]. The experiments on *ibm_strasbourg* were conducted in July 2025. Circuits were transpiled by *generate_preset_pass_manager* [61] with *optimization_level*= 1. No particular error mitigation techniques are set. We also run a benchmark with an ideal quantum simulator [58]. Exponentially increasing memory requirements limit the classical simulation of QAOA to roughly 30 variables. We note that our method of calculating parameters has no optimality guarantee and that better QAOA parameters might exist. Finally, we remark that like for quantum annealing, we decided against more advanced concepts like warm-starts [62] or constraint-preserving mixers [63] since we aim at testing the existing quantum architecture without method fine-tuning.

Steepest Descent. The steepest descent local search tries to improve the current solution by investigating its neighborhood. Here, the neighborhood is defined as all solutions having a Hamming distance of one to the current solution, which is equivalent to one-bit-flips. If the best solution in the neighborhood improves the objective value, it is accepted as the new current solution. Otherwise, the algorithm terminates. The main advantage of the steepest descent search is its fast runtime. A disadvantage is its inability to escape local minima. In our benchmark study, we employ the open-source implementation available in [64, 65].

Simulated Annealing. Simulated annealing is a popular classical heuristic. The name of this algorithm is based on its similarity to the process of annealing in metallurgy, which aims to harden a metal by controlled heating and cooling. In its general form, simulated annealing starts with a random solution. Then, simulated annealing iteratively considers a random solution in the neighborhood of the current solutions. If this solution improves upon the current objective value, it is accepted as the new current solution. Otherwise, it is still accepted as the new current solution with a probability decreasing with the number of iterations. By this methodology, the algorithm explores a large fraction of the solution space at the start, while at the end, objective-improving solutions are favored. The algorithm returns the solution with the best objective value [66]. Several parameters can be modified, for example, the definition of the neighborhood function and the acceptance probability function. In our computational study, we use an open-source implementation of simulated annealing [67, 68].

Tabu Search. Tabu search is a classical heuristic conceptually similar to steepest descent. Tabu search starts with a random solution, then explores the neighborhood of this solution and chooses the best solution in this neighborhood while putting the previous solution on a tabu list. Solutions on the tabu list are excluded from any neighborhood. The tabu list has a limited length and solutions are removed in a first-in-first-out manner. The tabu list allows the algorithm to escape local minima. Termination criteria can be runtime, number of steps or objective value. Adjustable parameters are the selection of the neighborhood function, the termination criterion and the length of the tabu list. In our computational study we use an open-source implementation of tabu search [69, 70].

Problem-specific heuristic. We also developed a problem-specific heuristic for the MinVola version of the portfolio optimization problem which does not require building the mathematical model. For an instance with n assets, it creates up to n feasible solutions, but does not guarantee feasibility. The heuristic starts with an empty portfolio and then increases the asset weight ω_i of some asset $i \in \{1, \dots, n\}$ by a fixed amount of $\delta > 0$. Then, it iteratively adds weight δ to the asset that reduces the portfolio variance the most while still satisfying constraint (1). In our case, we use $\delta = \frac{u_i}{2^d}$ to apply the same discretization as for the other methods. If the return constraint can not be satisfied, the algorithm increases the weight of the asset with the largest return, i.e., it moves towards return feasibility rather than terminating. The heuristic stops when the sum of weights is greater or equal to 1. Finally, the weights are normalized and the solution is checked for feasibility For a more detailed description and pseudocode of the problem specific heuristic, we refer to appendix C.

6 Computational Experiments

In this section, we benchmark the solution methods discussed in Section 5 on the portfolio optimization problem test set introduced in Section 2. The goal of our computational experiments is to quantify the potential of currently available quantum computers for portfolio optimization. Our study follows the general rules on good-practice for benchmarks from [71], which are relevance, reproducibility, fairness, verifiability and usability. Our code and data are publicly available at [72].

Benchmark procedure. The benchmark study was conducted on a standard notebook equipped with a 10-core, 16 GB RAM Intel Core i7 CPU. We compare the solution methods introduced in Section 5, that is, QAOA, quantum annealing, simulated annealing, steepest descent, tabu search and the problem-specific heuristic on the MinVola problem variant.
In the QUBO model (3), we set the penalty factors $\phi = \psi = 1000$ for violations of the return constraint (1) and the normalization constraint (2), respectively. We determine these factors by analyzing the magnitude of the objective and constraint terms. We detail the derivation of the penalty factors in appendix D, where we conduct a magnitude-analysis of the objective terms for the optimization problem.
With these penalty factors, a small absolute constraint violation of 0.01 in the constraints (1) or (2) will lead to a penalization of $1000 \cdot 0.01^2 = 0.1$ which is in the same order of magnitude as the objective value. Additionally, we choose the discretization coarseness in (4) as $d = 3$. This leads to $d + 1 = 4$ times as many variables in the discretized model as in the continuous model. Consequently, for an instance with n assets, the resulting QUBO has $4n$ variables. The value of $d = 3$ results from a trade-off between QUBO size and discretization precision.
To ensure fairness among the methods, we enforce a time limit of 60 seconds on the execution time of the algorithm on the device, i.e., the sampling time. We do not include the model construction into

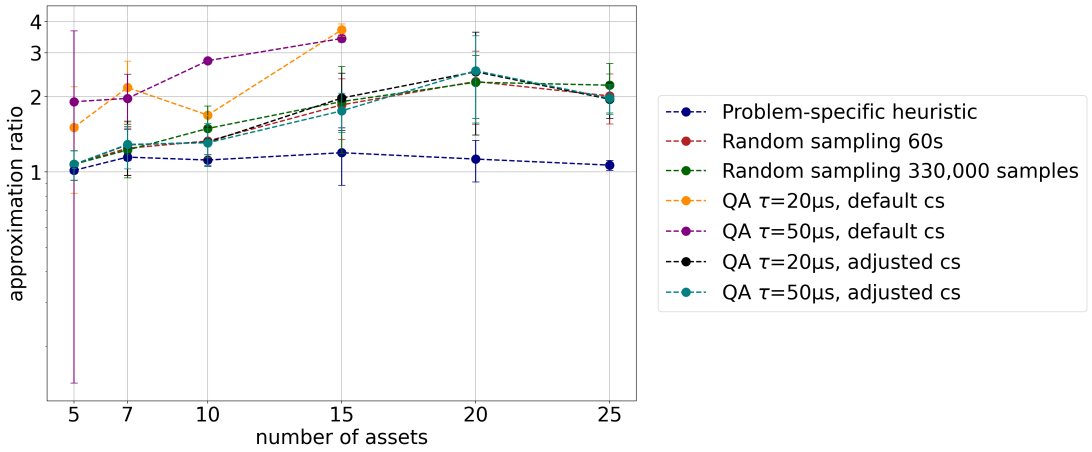
the time limit since the time for model construction varies depending on which library is used (like qiskit_optimization or dimod). Moreover, our process of first building the Quadratic Program and then transforming it into a QUBO could be sped up by directly building the QUBO, possibly in a fast programming language like C++. For the same reason, we do not include the result analysis time, i.e., calculating the QUBO objective values, which could be sped up by a faster implementation. We also do not include the embedding time into the time limit since the problem is always fully connected and thus, ideally, the embedding for a problem is calculated just once and reused for every other problem of the same size. Also, cloud connection, job submission and queueing times are not considered because we are interested in analyzing the capabilities of the quantum computer itself rather than access overhead. For the grid-search QAOA, we split up the 60 seconds into 30 seconds of training and 30 seconds of sampling with the best found parameters. We configure the size of the grid such that its execution time takes roughly 30 seconds. For the QAOA with optimized parameters, we spent a maximum of 52 seconds on training while the rest is used to exclusively sample with the best found parameters. This ensures that we have at least a minimum amount of time for sampling with high-quality parameters. Of course, the sampling results found during the training of QAOA are also considered in the final result. We remark that the accounting of runtime is deliberately favourable to the quantum methods since there is some infrastructure overhead involved like cloud connection, job submission and queueing. Finally, we note that a slightly smaller budget than 60 seconds was configured in the implementation as a safety margin to ensure that the limit is never exceeded.
After sampling within the boundaries of the time limit, we calculate the QUBO objective values of the entire sample. We take the 100 best solutions and execute a postprocessing step by scaling the weights such that they sum up to 1 in order to satisfy the normalization constraint (2). Then, we analyze these solutions with respect to feasibility and objective value for the original MinVola problem.
We then return the best found solution. This resembles a practical setting where one is usually interested in the best solution rather than statistical performance measures like expectation values or quantiles.
As our primary solution quality metric, we consider the *approximation ratio* which is defined as

$$\Theta := \frac{f_m}{f_{\text{opt}}} \geq 1.$$

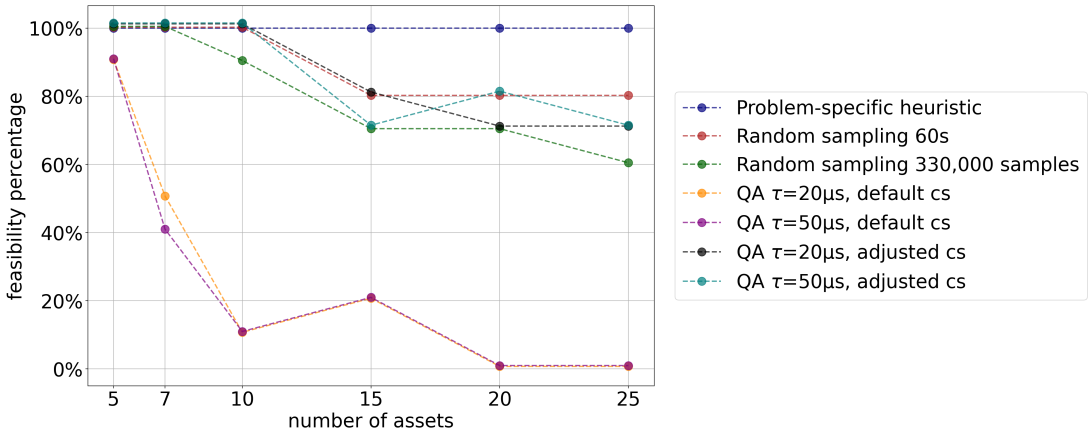
Here f_m is the objective value returned by the method m, and f_{opt} is the value of the optimal solution of the discretized version of the problem. which we calculate by an exact mathematical optimization solver. We have $f_m \geq f_{\text{opt}} \geq 0$ since MinVola is a minimization problem and the portfolio variance is always non-negative. Thus, a value of Θ close to 1 indicates near-optimal solutions.
In addition, we measure the *feasibility percentage*, which we define as the fraction of the 10 test instances per problem size for which the considered method returns at least a single feasible solution.
Finally, the *number of samples* measures how many samples are generated by the respective algorithm within the time limit.

Benchmark Results. We first discuss the results of quantum annealing, shown in Figures 2a, 2b and 2c. We observe that quantum annealing with the default calculation of the chain strength cs yields the worst approximation ratio (Figure 2a) and the lowest feasibility percentage (Figure 2b) in the red and pink lines. In particular, even random sampling has a lower average approximation ratio on the same number of samples while finding more feasible solutions (compare green and yellow lines in Figures 2a, 2b, 2c).
After observing high chain break fractions, we tested different chain strength parameters. Since all QUBO instances of equal size are fully connected and embedded using the same clique-embedding strategy, the resulting embeddings exhibit identical chain structures and chain lengths across instances of the same size. Empirically, the observed chain break fractions were nearly identical across different problem instances of equal size, indicating that chain break behavior is dominated primarily by problem size and embedding geometry rather than by instance-specific QUBO coefficients. For problems of $5, 7, 10$ and 15 assets we set $cs = 3$, for 20 assets we set $cs = 4$ and for 25 assets we set $cs = 5$. These parameters result in chain break fractions in the desired range of 0% to 5% as recommended by D-wave during a private correspondence. We would like to point out that such chain strength tuning can only improve the quantum annealing results, but it adds a bias to the reported performance estimate. The results of quantum annealing with the customized chain strengths are shown in the black and gray lines in Figures 2a, 2b and 2c. We observe that quantum annealing with adjusted chain strength is roughly on par with random sampling in terms of both feasibility percentage and approximation ratio. Nevertheless, we observe in Figures 2a and 2b that quantum annealing performs significantly worse than the problem-specific heuristic. We conjecture that the high density of the QUBO problem is a reason for the suboptimal performance of

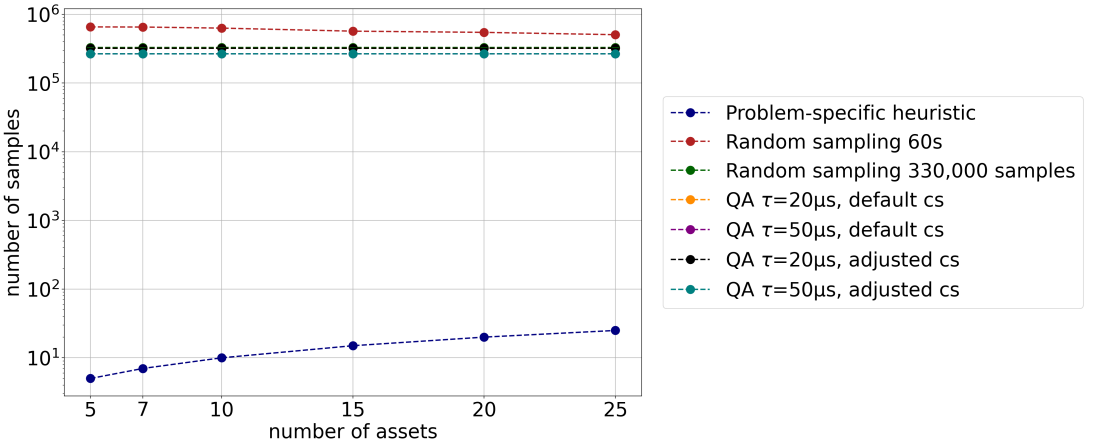
quantum annealing. Embedding the required all-to-all connectivity leads to long chains of qubits for each variable. Emphasizing the large embedding overhead, we note that the largest problem size that can still be embedded on the quantum annealing processor with $4,400+$ physical qubits has 25 assets, which corresponds to 100 QUBO variables. Moreover, varying the annealing time between 20 μs and 50 μs did not significantly improve the results, compare pink versus red and grey versus black in Figures 2a and 2b. We remark that we have conducted additional experiments with annealing times of 1 μs and 5 μs. Those results did not differ significantly from the results with annealing times of 20 μs and 50 μs. Finally, in Figure 2c, we observe that the number of samples generated in 60 s is independent of the problem size. From this, we conclude that the runtime of quantum annealing does not significantly increase with problem size, which might lead to an advantage of future annealing hardware for very large instances.
Next, we turn to the results for QAOA, shown in Figures 3a, 3b, 3c. Similar to quantum annealing, QAOA often falls short on finding feasible solutions, which results in decaying feasibility percentages in Figure 3b. Even when QAOA returns feasible solutions, their approximation ratios quickly turn suboptimal when increasing the problem size. We observe approximation values of ≥ 2 for problems with 20 or more assets in Figure 3a. In general, QAOA does not perform significantly better than random sampling with respect to feasibility and approximation ratio. Compared to the problem-specific heuristic, all QAOA configurations deliver lower feasibility percentages and worse approximation ratios. The QAOA configuration with the best results is the 1-layer linear-ramp (red lines) which returns a lower approximation ratio with a higher feasibility percentage on a similar number of samples than the remaining QAOA configuration. Moreover, we observe that increasing the number of layers worsens the performance although, in theory, the performance improves with the number of layers. Other experiments with 2 and 4 layers affirm this observation while we do not show their results here for clarity. We attribute this paradox mainly to two reasons. First, more layers increase the circuit execution time. Consequently, less shots can be executed in the same time, which can be seen in Figure 3c. Second, the effect of noise increases with the circuit size. A reason for the decaying performance of QAOA with larger problem instances is the rapidly increasing number of swap gates that are required to transpile the QAOA circuit with all-to-all-connectivity to the quantum computer with limited connectivity. For example, for 20 assets, the circuit depth of 1-layer QAOA increases from 161 to $\approx 11,000$ during transpilation.
The results for the classically simulated QAOA are only reported in appendix E, as the largest instance which can still be simulated has only 7 assets. Finally, we remark that we did not run QAOA experiments in which we optimize the parameters on the hardware since the communication and queuing times are prohibitively large which is highly impractical for a parameter optimization loop. We remark that IBM offers a session mode which reduces queuing, however, it drastically consumes more hardware access time than for the jobs themselves.
Lastly, we compare the results for classical heuristics in Figures 4a, 4b and 4c. All open-source implementations (steepest descent, simulated annealing and tabu search) are executed with default settings. In general, all heuristics are able to find feasible solutions for nearly all problem instances, see Figure 4b. Only tabu search does not return feasible solutions for all instances with 800 assets. On the other hand, random sampling failed to return any feasible solutions starting from 40 assets. The approximation ratios of simulated annealing, steepest descent and tabu search exceed the value of 2 starting at 50 assets and the value of 10 at 250 or more assets. The problem-specific heuristic performs best with respect to the average approximation ratio. The approximation ratios of the other heuristics are comparable despite differing significantly in the numbers of samples (Figure 4c). We remark that all heuristics rely on objective evaluation which can take a significant amount of time on large instances. This explains the decreasing number of samples for larger problem instances. For example, for 500 assets, the problem-specific heuristic is only able to return roughly 100 out of 500 possible solutions in 60 s. Thus, quantum methods might be beneficial for even higher asset numbers since they require classical objective evaluation only during the sample evaluation process and not during sampling itself. Nevertheless, the performance of the problem-specific heuristic sets the bar high for a possible quantum advantage.
In Figures 5a, 5b and 5c, we compare the most promising approaches from quantum annealing, QAOA and classical heuristics in order to rank them. In particular, we consider quantum annealing with 50 µs annealing time and adjusted chain strength, the single-layer QAOA with linear ramp parameters and the problem-specific heuristic.
We observe that the problem-specific heuristic has a larger feasibility percentage and a better approximation ratio than the best configurations of both QAOA and quantum annealing. Furthermore, both QAOA and quantum annealing perform roughly as good as random sampling. Notably, random sampling generates significantly more solutions in 60 s (Figure 5c). Finally, we conclude that quantum annealing slightly outperforms QAOA since quantum annealing generates more samples, finds more feasible solu-



(a) quantum annealing approximation ratio

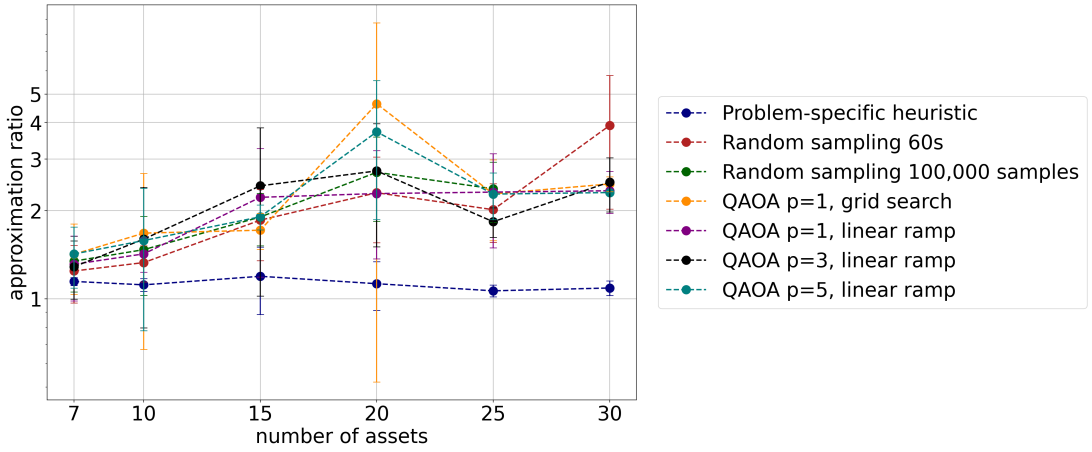


(b) quantum annealing feasibility percentage

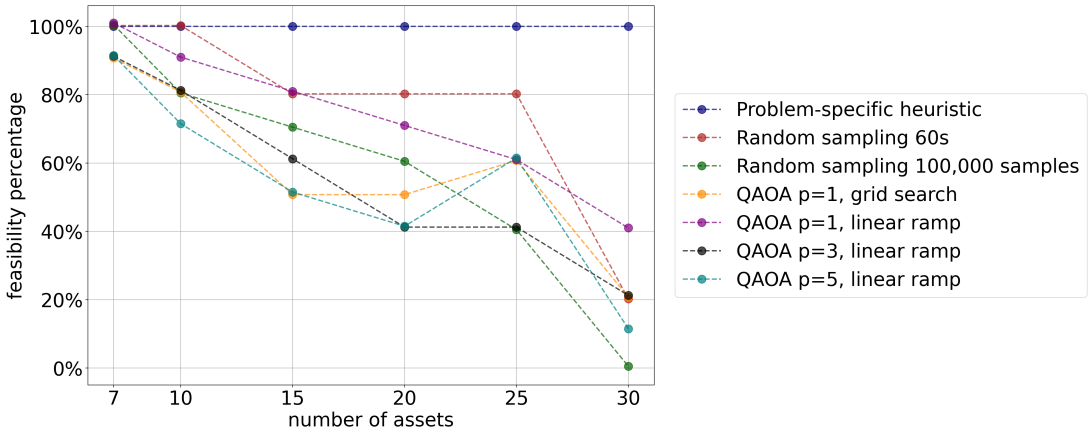


(c) quantum annealing number of samples

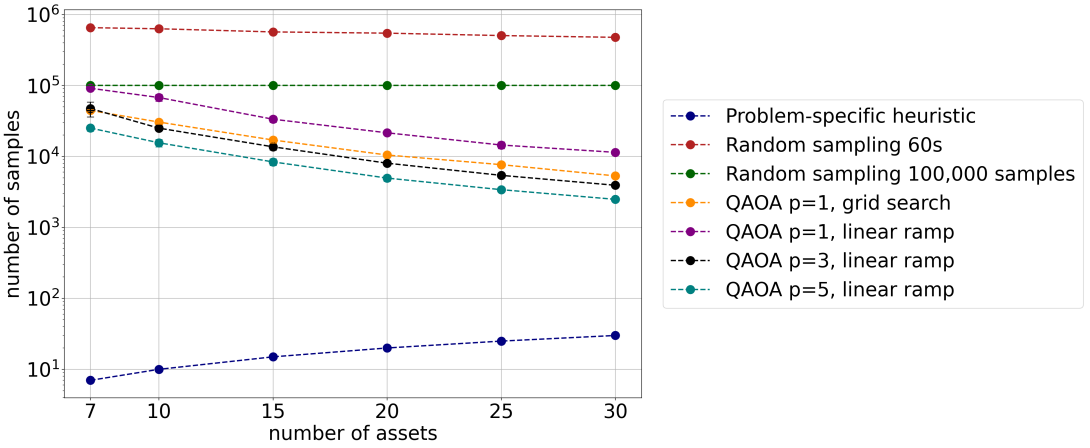
Figure 2: We report the approximation ratio, feasibility percentage and number of samples for quantum annealing. We compare against the problem-specific heuristic and random sampling as baselines. Data points are averages over 10 instances and error bars show the empirical standard deviation. Additionally, distributions of the approximation ratios over the 10 instances can be viewed in the appendix chapter F. Missing data points in the approximation ratio are due to the absence of feasible solutions.



(a) QAOA approximation ratio

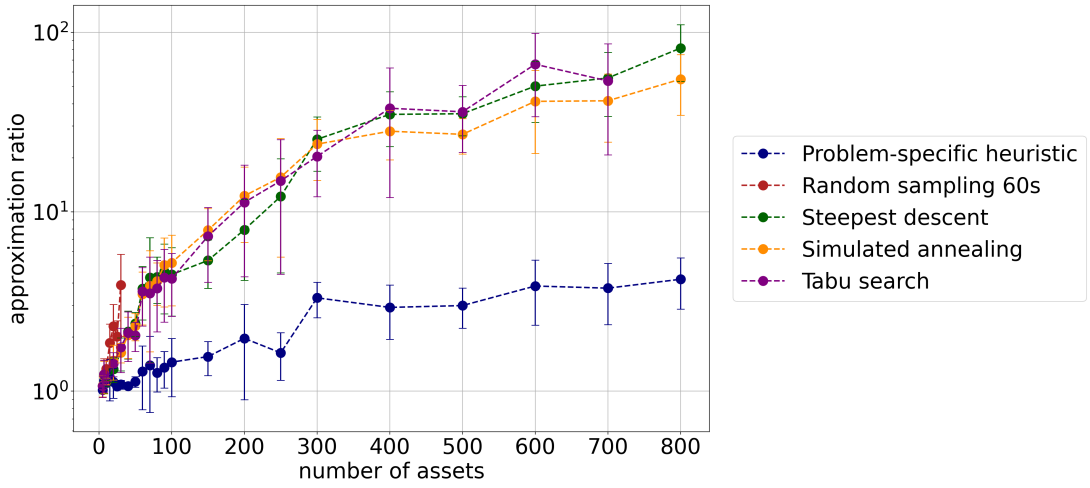


(b) QAOA feasibility percentage

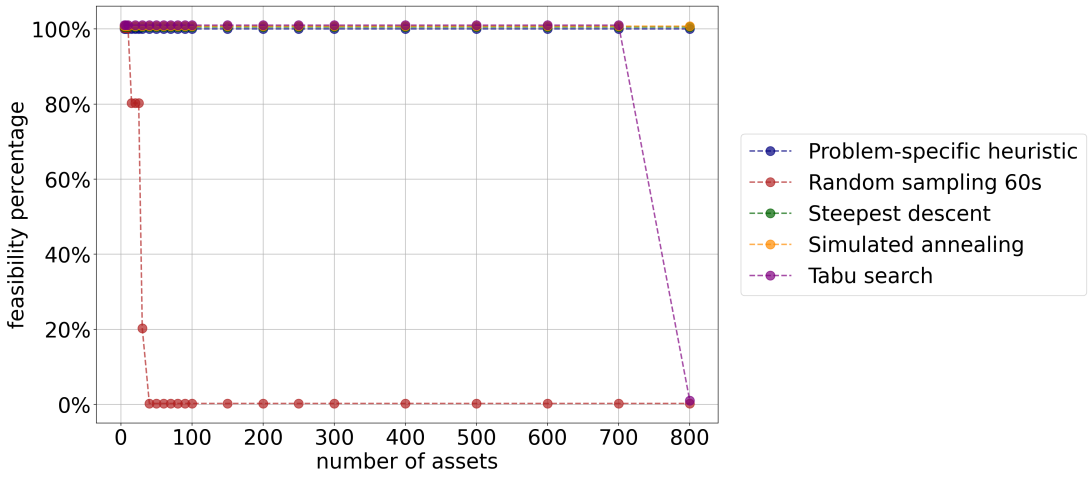


(c) QAOA number of samples

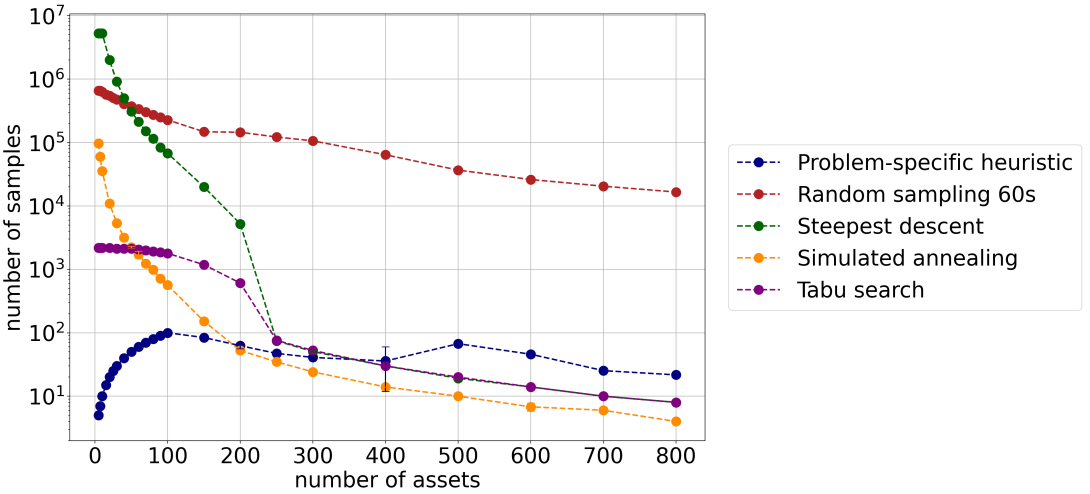
Figure 3: We report the approximation ratio, feasibility percentage and number of samples for QAOA. We compare against the problem-specific heuristic and random sampling as baselines. Data points are averages over 10 instances and error bars show the empirical standard deviation. Additionally, distributions of the approximation ratios over the 10 instances can be viewed in the appendix chapter F. Missing data points in the approximation ratio are due to the absence of feasible solutions.



(a) heuristics approximation ratio

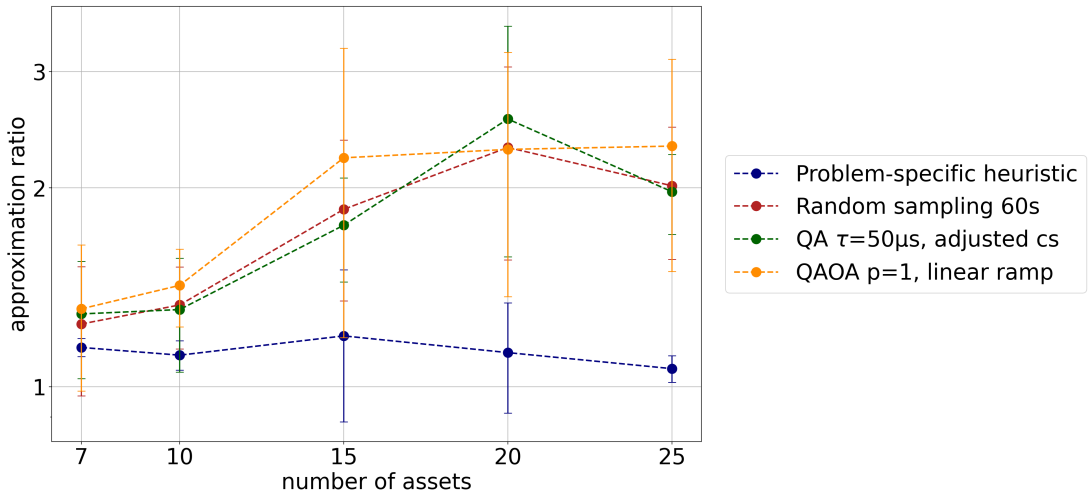


(b) heuristics feasibility percentage

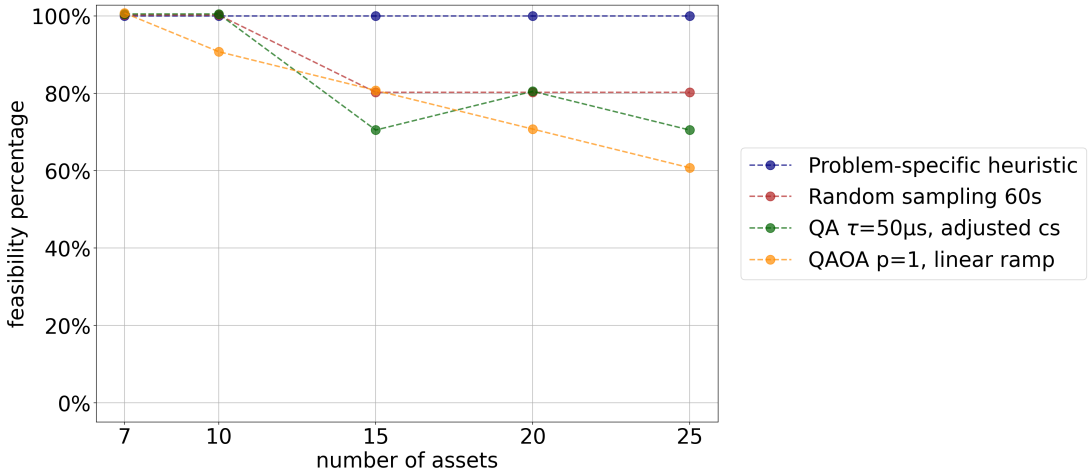


(c) heuristics number of samples

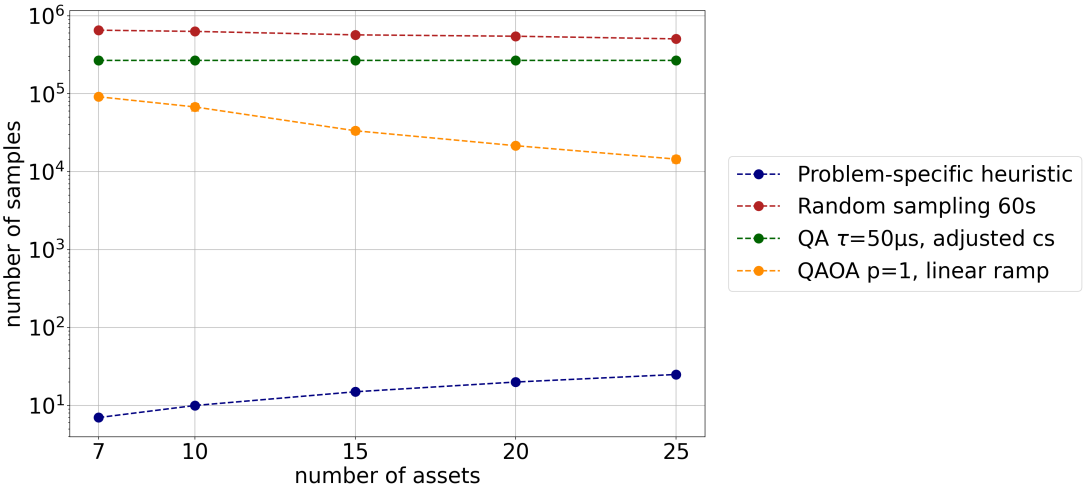
Figure 4: We report the approximation ratio, feasibility percentage and number of samples for the heuristics. We compare against the problem-specific heuristic and random sampling as baselines. Data points are averages over 10 instances and error bars show the empirical standard deviation. Additionally, distributions of the approximation ratios over the 10 instances can be viewed in the appendix chapter F. Missing data points in the approximation ratio are due to the absence of feasible solutions.



(a) Best methods: approximation ratio



(b) Best methods: feasibility percentage



(c) Best methods: number of samples

Figure 5: Comparison of the best performing configurations from our computational study. The best performing configurations are the Problem-specific heuristic, quantum annealing with annealing time of $50\mu s$ and adjusted chain strengths as well as QAOA with 1 layer and linear-ramp parameters. For comparison purposes, random sampling for 60 seconds is also visualized. We report the approximation ratio (a), the feasibility percentage (b) and the number of samples generated during the given time limit of 60 seconds (c). Data points are averages over 10 instances and error bars show the empirical standard deviation.

tions and those solutions are also of slightly higher quality. The difference in the number of generated samples would get larger for increasing problem size since the QAOA circuit length and thus its execution time grows with the number of variables whereas the annealing time stays constant.

7 Conclusion

In this work, we conducted an extensive benchmark of quantum computing for portfolio optimization. Recently, several works suggested portfolio optimization as a suitable candidate for a possible quantum advantage. Our experiments, however, highlight the challenges of achieving such an advantage in practice. We consider a variance-minimizing variant of portfolio optimization which we have shown to be more difficult for classical optimizers than return-maximizing or multi-objective formulations. We then compared both classical and quantum methods on 270 problem instances from real-world stock data. In our benchmark, we impose a time limit of 60 seconds on the sampling and result analysis to ensure fairness among the methods. Our main conclusion is that classical heuristics like simulated annealing, steepest descent, tabu search and a problem-specific heuristic clearly outperform QAOA and quantum annealing regarding solution feasibility and quality. Regarding classical exact optimizers, Gurobi and SCIP differ significantly in their solution times. Here, Gurobi is more than 1000 times faster for large problem instances, solving problems with 1000 assets in the order of seconds. Comparing quantum methods, we observed that quantum annealing slightly outperformed the gate-based QAOA in terms of feasibility and quality. However, this superiority of quantum annealing only appeared after fine-tuning chain strengths. Because the adjusted chain strengths improve feasibility and approximation quality, any residual overfitting would bias the results in favor of quantum annealing and therefore does not affect the main conclusion of this study. While being outperformed by classical heuristics, QAOA and quantum annealing also did not clearly differ from random sampling within the time limitation of 60 seconds. Here, we emphasize that we observed a clear difference in the distribution of objective values when comparing QAOA and quantum annealing to random sampling. However, in this work we considered only the best solution instead of statistical measures of the solution distribution. We conjecture the all-to-all connectivity of the problem to be the main reason for the poor performance of QAOA and quantum annealing. Dense problems generate a large qubit overhead during embedding and a large gate overhead during transpilation. Finally, we remark that the considered problem variant is one of the simpler formulations of portfolio optimization. For more complicated problem variants with additional constraints and variables, further experiments are required to study the performance of quantum methods.

Acknowledgements

We thank D-Wave for their efforts in reviewing our experiments and for their recommendations on parameter settings. This research was conducted within the Bench-QC project, a lighthouse project of the Munich Quantum Valley initiative, which is supported by the Bavarian state with funds from the Hightech Agenda Bayern Plus.

Appendix

A Generating Stock Data

For our benchmark, we create portfolio problem instances from real world stock data. We use the Python package yfinance [40] to retrieve historical NASDAQ closing prices $p_{i,t}$ of asset i at time t. From the price data, we calculate the *daily asset returns*

$$rd_{i,t} = \frac{p_{i,t}}{p_{i,t-1}} - 1 \quad \forall i \in N,\ t \in T$$

and the *average daily asset returns*

$$rav_i = \frac{1}{|T|} \sum_{t \in T} rd_{i,t} \quad \forall i \in N.$$

For the portfolio optimization problem, we work with *annualized asset returns*, which are defined by

$$r_i \coloneqq \left(\prod_{t \in T} (1 + rd_{i,t}) \right)^{\frac{252}{|T|}} - 1 \quad \forall i \in N.$$

The value of 252 is the average yearly amount of business days at the stock exchange. The estimated *annualized covariances* for all asset combinations of asset (i, j) are calculated by

$$\sigma_{ij} \coloneqq \frac{252}{|T|} \sum_{t \in T} (rd_{i,t} - rav_i)(rd_{j,t} - rav_j) \quad \forall i, j \in N. \tag{6}$$

The covariance matrix is used in its raw form without regularization because of the large number of observations in our test data set of 4 years. Now, the annualized asset returns r_i and the annualized asset covariances σ_{ij} are used to create instances of the portfolio optimization problem.

B Efficient Frontier

Our argument for replacing the $\geq$ with $=$ in model MinVola relies on the *efficient frontier* together with the selection of ϵ. According to the Markowitz Portfolio Theory [35], when displaying the minimum portfolio variance for given portfolio returns, a hyperbola arises [73]. The upper part of this hyperbola is called the efficient frontier. We visualize this concept by a schematic illustration in Fig. 6. The efficient frontier is a hyperbola where the horizontal axis represents risk (standard deviation) and the vertical axis represents expected return.

The portfolio which produces the leftmost point on this hyperbola is the global variance-minimal portfolio p_* with return and variance (μ_*, σ_*^2). Its return splits up the curve into two parts, $\mu < \mu_*$ and $\mu \geq \mu_*$. The upper part of this hyperbola consists of efficient portfolios whereas the lower part consists of inefficient portfolios, because for these, a portfolio with a higher return and the same variance also exists. Now, if we select $\epsilon \geq \mu_*$, the minimum variance portfolio for a larger-than-ϵ return has exactly return ϵ. In our benchmark test set we select ϵ as the 70 % quantile of its asset returns. Thus, we do not explicitly enforce $\epsilon \geq \mu_*$. Nonetheless, we assume this to be mostly the case, because with the 70 % quantile, we have a higher-than-average returning portfolio. We checked this assumption by solving the "$\mu \geq \epsilon$" quadratic program for all instances of our benchmark set. We observe that 93, 75 % of the variance-minimal portfolio displayed $\mu = \epsilon$. Furthermore, we observe that this assumption tends to be more often violated on small instances. When considering only instances that we used in the quantum benchmarks, i.e., from 5 to 30 assets, $\mu = \epsilon$ holds only with a percentage of 80 %. To further analyze the magnitude of the incurred inaccuracy, we calculate the difference in variance between optimizing with "$\geq$" and "$=$" for the instances where $\mu = \epsilon$ did not hold. The mean relative difference amounts to 0.025. To also report absolute values, the mean absolute difference in optimal variances for instances from 5 to 30 assets turns out to be 0.0015, the maximum difference is 0.004. As a result of these small deviations, we argue that it is indeed reasonable to replace $\geq$ by $=$, as in most cases, the optimum solution to the "$\geq$" quadratic program is attained at $\mu = \epsilon$. For instances where this is not the case, the incurred error in optimum variance is small. Replacing $\geq$ by $=$ also benefits the QUBO model because no additional slack variables have to be introduced, which in turn would have to be discretized and thus would heavily increase the qubit count and further reduce the accessible instance sizes. We acknowledge that this is a trade-off between model-size and modelling accuracy. The same argumentation holds for a one-sided penalty construction, which introduces additional non-quadratic structure or auxiliary variables.

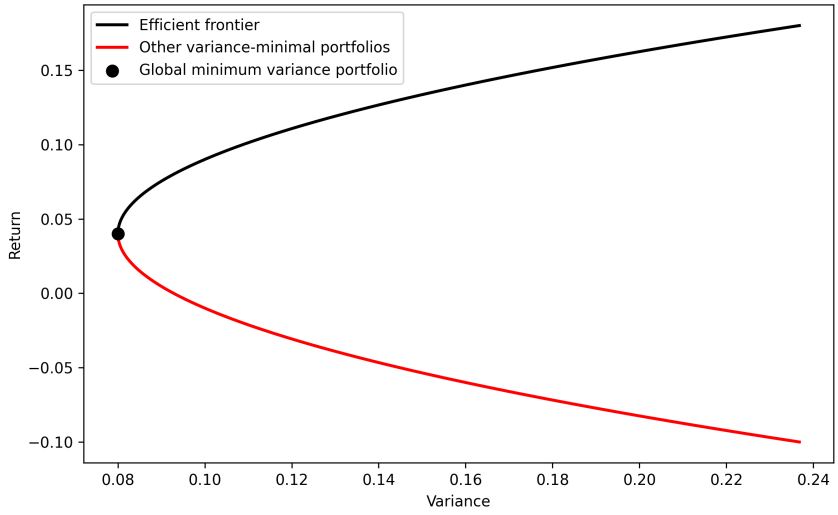


Figure 6: Visualization of the concept of the efficient frontier. The axes have arbitrary units.

C Pseudocode Problem-specific Heuristic

The Problem-specific heuristic for the MinVola problem formulation with $n \in \mathbb{N}$ possible assets generates up to n feasible solutions.

Algorithm 1 MinVola - Problem-specific Heuristic

Let $\mu_i \in \mathbb{R} \leftarrow$ expected return for asset $i \in \{1, \ldots, n\}$
Let $u_i \in (0, 1] \leftarrow$ upper bound of asset weight for asset $i \in \{1, \ldots, n\}$
Let $\sigma_{ij} \in \mathbb{R} \leftarrow$ covariance of returns of assets $i, j \in \{1, \ldots, n\}$
Let $\epsilon \in \mathbb{R} \leftarrow$ minimum required portfolio return
Let $\delta \in (0, 1] \leftarrow$ constant weight parameter that gets added to the samples
Initialize `Sols` $\leftarrow$ empty list of solutions
for $i \leftarrow 1$ **to** n **do**
    Initialize $x \leftarrow [0, \ldots, 0]$ empty asset weight vector of length n
    $x_i \leftarrow \delta$ add weight to asset i
    while $\sum_{k=1}^{n} x_k < 1 - \frac{\delta}{2}$ **do**
        $x \leftarrow$ AddNewAssetWeightSteepestDesc$(x, \mu, ub, \sigma, \delta, \epsilon)$
    end while
    $x \leftarrow$ NormalizeAssetWeights(x)
    Add x to `Sols`
end for
return `Sols`

function AddNewAssetWeightSteepestDesc$(x, \mu, ub, \sigma, \delta, \epsilon)$
    Let big_M $\leftarrow 1000$
    Let small_M $\leftarrow -1000$
    Initialize $Variances \leftarrow [\,]$ variances
    Initialize $Rets \leftarrow [\,]$ returns
    for $j \leftarrow 1$ **to** n **do**
        Let $x' \leftarrow$ copy of x
        $x'_j \leftarrow x'_j + \delta$
        if $x'_j \leq ub_j$ **then**
            $r \leftarrow \frac{\sum_{k=1}^{n} x'_k \mu_k}{\sum_{k=1}^{n} x'_k}$ calculate normalized return
            Append r to $Rets$
            if $r \geq \epsilon$ **then**
                v $\leftarrow \sum_{k=1}^{n} \sum_{l=1}^{n} x'_k \sigma_{kl} x'_l$ calculate return variance
                Append v to $Variances$
            else
                Append big_M to $Variances$
            end if
        else
            Append small_M to $Rets$
            Append big_M to $Variances$
        end if
    end for
    if $\min(Variances) =$ big_M **then**
        $j^* \leftarrow \arg\max Rets$
    else
        $j^* \leftarrow \arg\min Variances$
    end if
    $x_{j^*} \leftarrow x_{j^*} + \delta$
    return x
end function

function NormalizeAssetWeights(x)
    sum_of_weights $\leftarrow \sum_{i=1}^{n} x_i$
    for $i \leftarrow 1$ **to** n **do**
        $x_i \leftarrow x_i \cdot$ sum_of_weights
    end for
    return x
end function

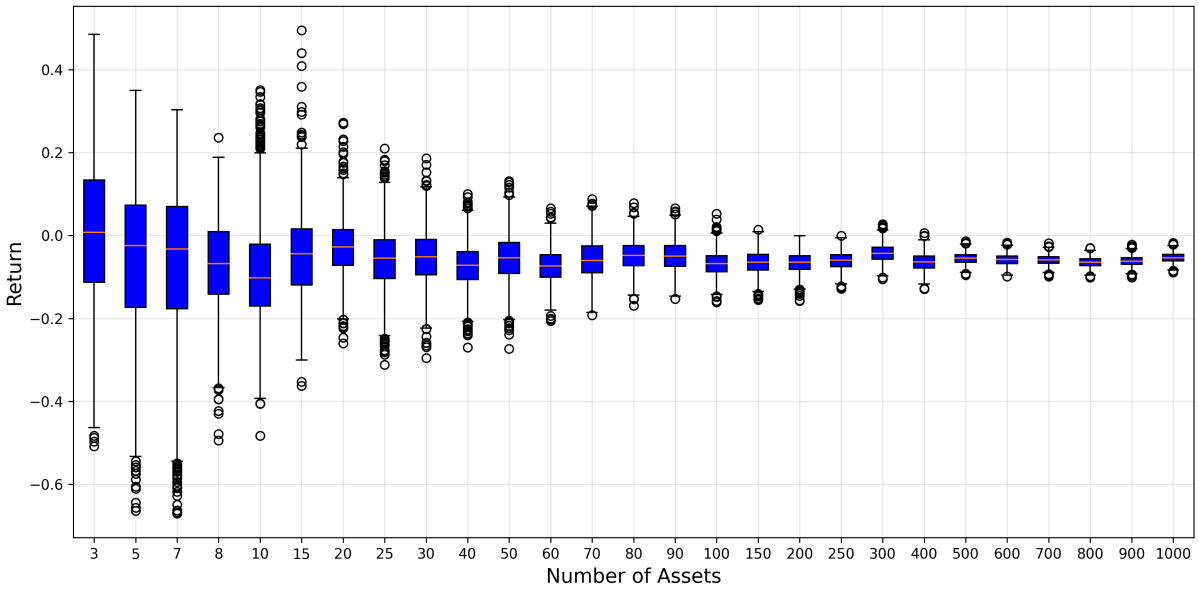


Figure 7: Distribution of returns of random portfolios for different instance sizes. For each instance size, we sampled 100 random portfolios for each of the 10 problem instances in our test data set.

D Selection of Penalty Factors

In the QUBO model (3), we set the penalty factors $\phi = \psi = 1000$ for violations of the return constraint (1) and the normalization constraint (2), respectively. Thus, an absolute constraint violation of 0.01 leads to a penalization of $1000 \cdot 0.01^2 = 0.1$. The goal is to make constraint violations unfavorable by increasing the objective value. Thus, we require that a constraint violation of 0.01 is small compared to the typical right-hand side of (1) and (2) and, at the same time, that a penalization of 0.1 is comparable to a typical objective value of MinVola. Clearly, 0.01 is small compared to the right-hand-side 1 of the normalization constraint (2). To analyze the domain of the right-hand-side of the return constraint (1), i.e. the 70%-quantile of the return $\mu(\omega)$, and the domain of the objective, i.e. the variance $\sigma^2(\omega)$, we calculate the returns and variances for 100 random portfolios for each of the 10 instances for each problem size (asset number). Thus, we analyze 28,000 random portfolios in total. The results are visualized in the boxplots in Fig. 7 and Fig. 8.

The boxplots visualize the statistical distribution of numerical data points by showing the median (red line), the 25th and 75th percentiles (blue box boundaries), the range of non-outlier values within 1.5 times the interquartile range (whiskers), and individual outlier points outside the whiskers. From this data, we estimate the typical magnitudes of portfolio returns and variances. The 75th percentiles (upper boundaries) of the portfolio returns are mostly negative and take values in the range from -0.1 to 0.1. We see that a return constraint violation of 0.01 is indeed small compared to a typical absolute value of ϵ, especially for instances with a small number of assets. These violations happen with decreasing frequency with increasing problem size. In particular, for $n \leq 30$ assets, which is the maximum number of assets that we are able to execute on a quantum computers, these frequently appearing high violations are penalized heavily, which is desired.

For the variances, the number of outliers is larger than for the returns. Some outliers have values above $10,000$ (e.g. for 40 assets). This is due to the high variance of the return variances of the individual assets which can cause a huge return variance in a random portfolio. Nonetheless, all medians have values below 0.35. Moreover, the optimal solutions to the instances show decreasing variances with increasing problem sizes, all below 0.5. This effect is visualized in Fig. 9. Thus, a penalty of 0.1 for a return-constraint violation of 0.01, is in a comparable range to the objective function as desired.

E Benchmark Study Results: Simulated QAOA

As mentioned in Section 5, a classically simulated QAOA, e.g. with AerSimulator [58], can only be executed for up to around 30 variables. As a result, the largest problem size that can be run with our configuration on the simulator is 7 assets, requiring 28 variables. For $3, 5$ and 7 assets we show the results

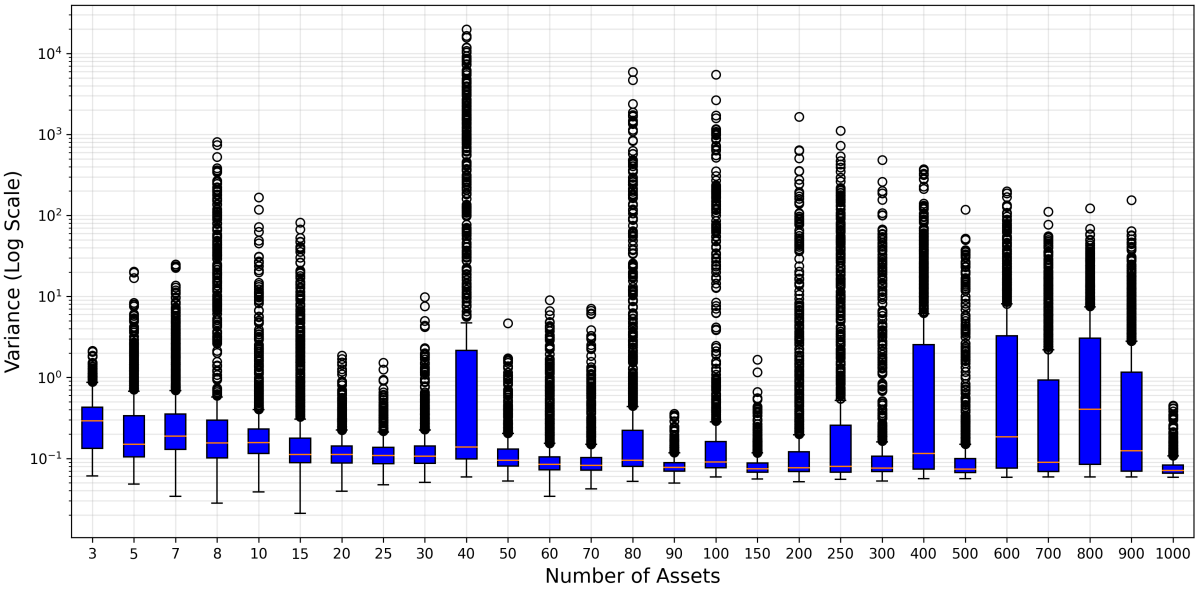


Figure 8: Distribution of variances of random portfolios for different instance sizes. For each instance size, we sampled 100 random portfolios for each of the 10 problem instances in our test data set.

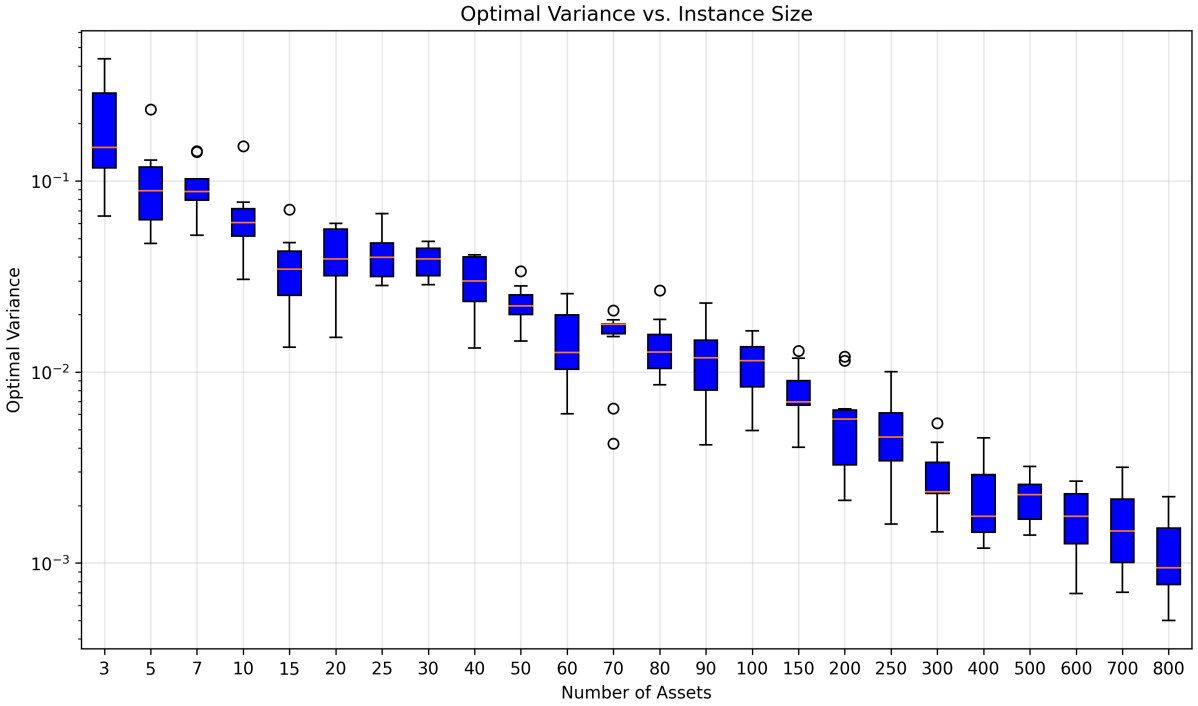


Figure 9: Distribution of optimal variances of the 10 discretized instances of each problem size in the benchmark data set.

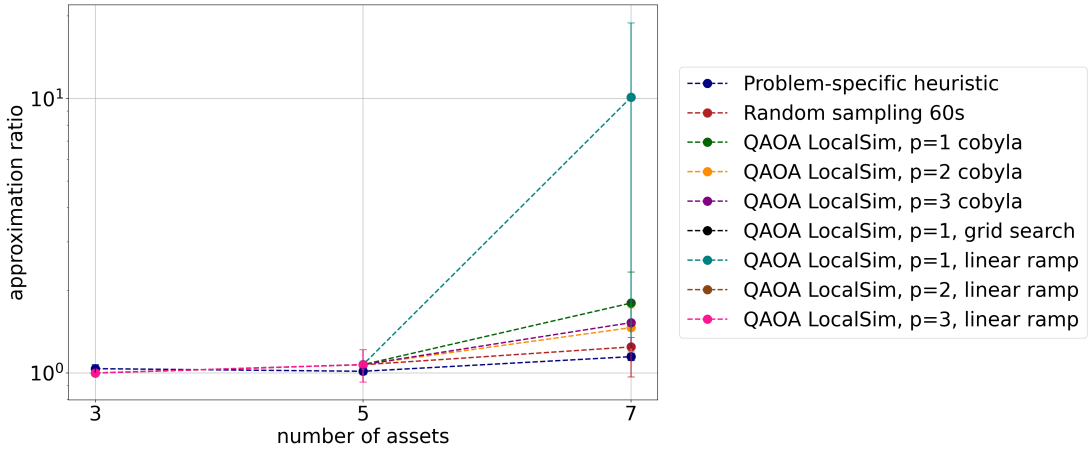
in Fig. 10. Therein, we compare different configurations of the locally simulated QAOA against random sampling and the problem-specific heuristic. We differentiate between the number of layers $p = 1, 2, 3$ and the way of optimizing the QAOA parameters. First, we use the COBYLA optimizer to optimize the parameters classically. Second, we perform a grid search and third, we initialize the parameters with the linear ramp formula. For further details on the grid search and the linear ramp, we refer to the QAOA section 5.

F Benchmark Study Results: Distribution of Approximation Ratios

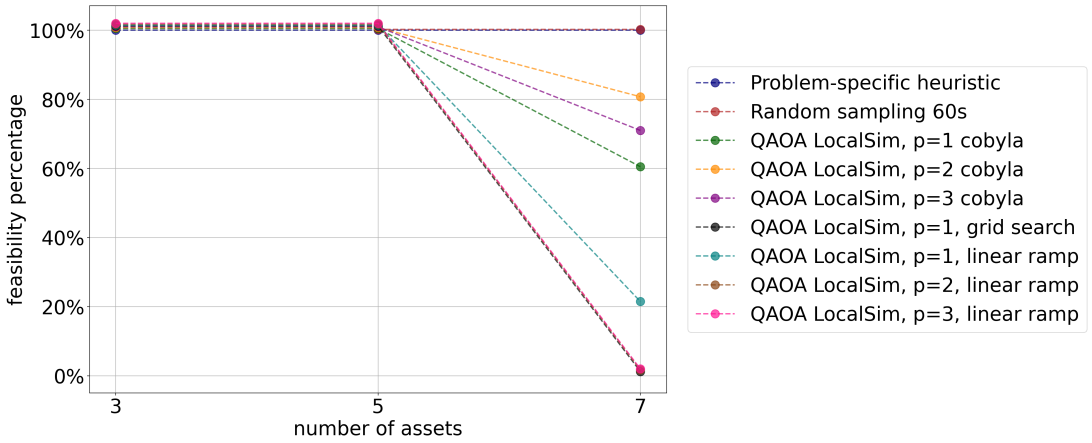
In the result section 6, we show the mean and empirical standard deviation of the achieved approximation ratios. In this appendix, we additionally visualize the distributions by boxplots in Fig. 11.

G Benchmark Study Results: Approximation Ratios With Respect to Continuous Problem

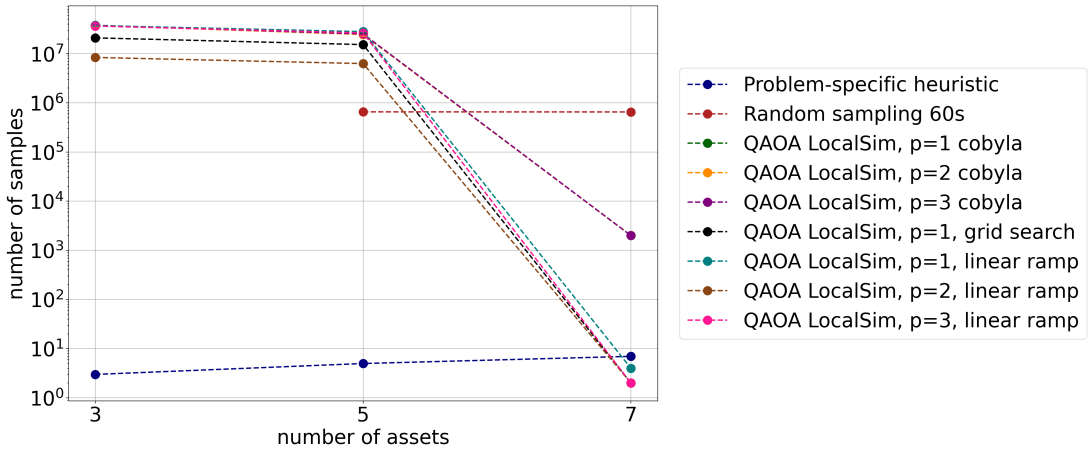
In section 6, we report the approximation ratios based on the objective value of the optimal solution to the discretized formulation of the MinVola problem. In Fig. 12, we additionally report the approximation ratios with respect to the optimal solution to the continuous MinVola problem. We observe similar results for the continuous case as for the discretized case. This similarity is due to the small difference between the optimal solution values of the discretized and the continuous problem, which further justifies the selection of the discretization factor of $d = 3$.



(a) Simulated QAOA approximation ratio

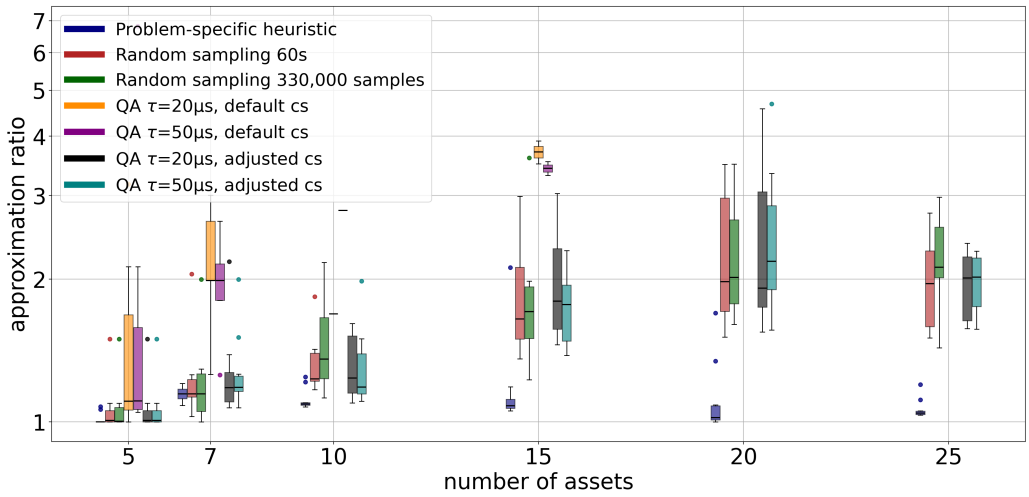


(b) Simulated QAOA feasibility percentage

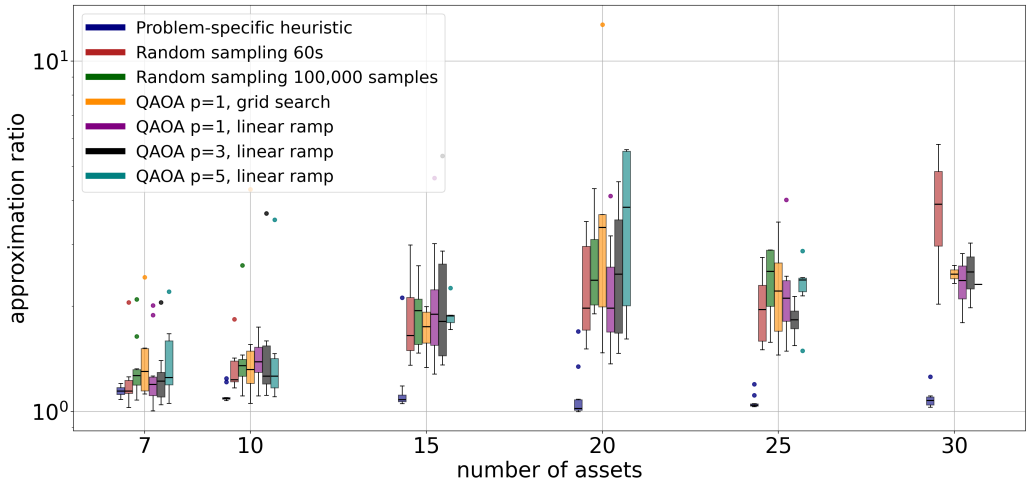


(c) Simulated QAOA number of samples

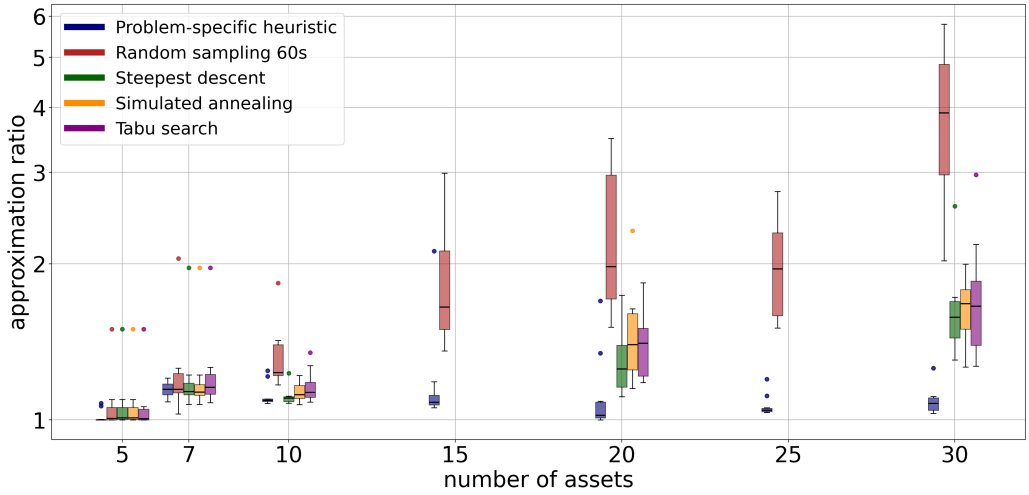
Figure 10: Approximation ratio, feasibility percentage and number of samples for the simulated QAOA. We compare against the problem-specific heuristic and random sampling as baselines. Data points are averages over 10 instances and error bars show the empirical standard deviation. Missing data points in the approximation ratio are due to the absence of feasible solutions.



(a) Boxplots of the quantum annealing approximation ratios of solving the 10 instances of each problem size in the benchmark data set.

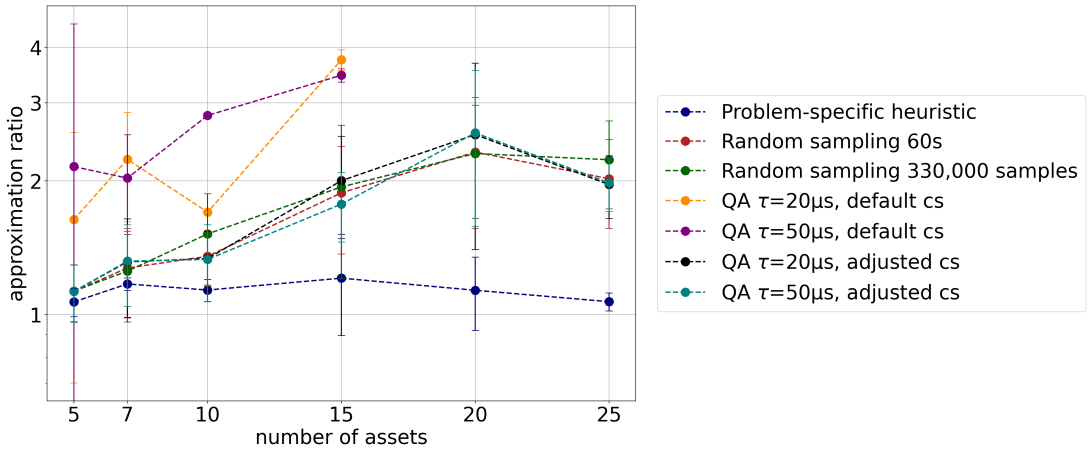


(b) Boxplots of the QAOA approximation ratios of solving the 10 instances of each problem size in the benchmark data set.

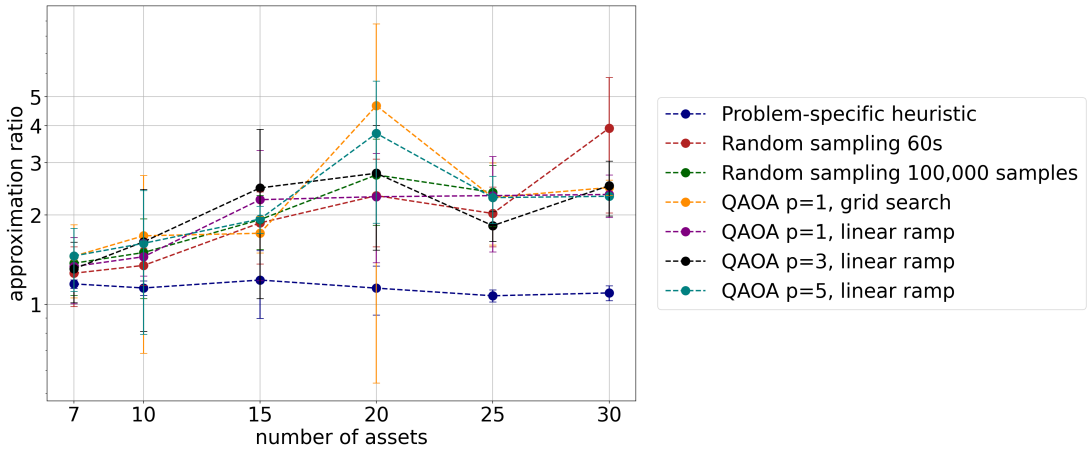


(c) Boxplots of the heuristics approximation ratios of solving the 10 instances of each problem size in the benchmark data set.

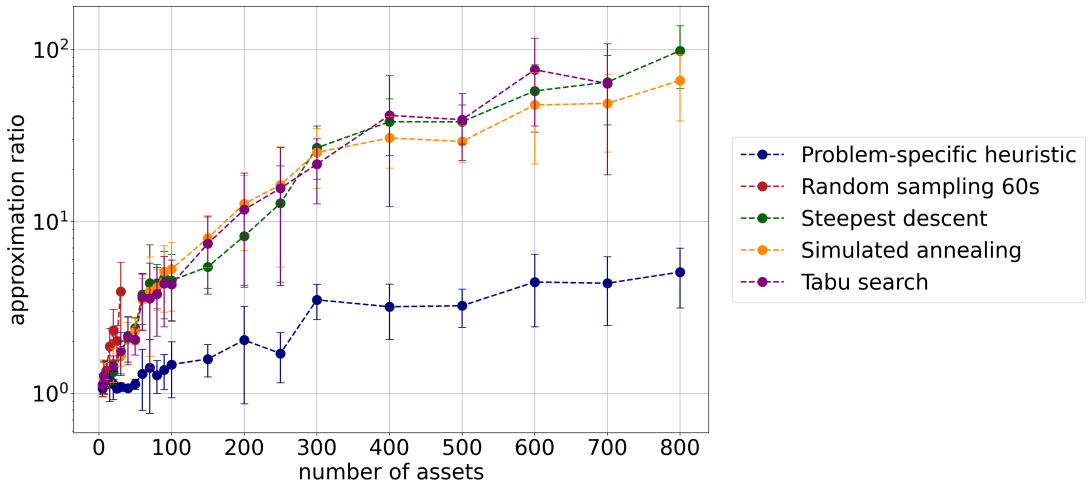
Figure 11: Boxplots of the approximation ratios of quantum annealing, QAOA, and the heuristics. They visualize the median (black lines inside the boxes), the 25th and 75th percentiles (box boundaries), the range of non-outlier values within 1.5 times the inter-quartile range (whiskers), and individual outlier points outside the whiskers.



(a) Quantum annealing approximation ratios w.r.t. the continuous MinVola problem.



(b) QAOA approximation ratios w.r.t. the continuous MinVola problem.



(c) Heuristics approximation ratios w.r.t. the continuous MinVola problem.

Figure 12: The approximation ratios of quantum annealing, QAOA, and the heuristics, where f_{opt} is selected as the optimal solution of the continuous version of the Minvola problem.

References

[1] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*. Vol. 21. Algorithms and Combinatorics. Berlin, Heidelberg: Springer, 2018. URL: https://link.springer.com/10.1007/978-3-662-56039-6 (visited on 22/09/2025).

[2] Thorsten Koch, Timo Berthold, Jaap Pedersen and Charlie Vanaret. "Progress in mathematical programming solvers from 2001 to 2020". In: *EURO Journal on Computational Optimization* 10 (2022), p. 100031. DOI: https://doi.org/10.1016/j.ejco.2022.100031.

[3] Michael Jünger, Thomas M. Liebling, Denis Naddef, George L. Nemhauser, William R. Pulleyblank, Gerhard Reinelt, Giovanni Rinaldi and Laurence A. Wolsey, eds. *50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art*. Berlin, Heidelberg: Springer, 2010. URL: http://link.springer.com/10.1007/978-3-540-68279-0 (visited on 22/09/2025).

[4] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp M. Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin and Yuji Shinano. "MIPLIB 2017: Data-Driven Compilation of the 6th Mixed-Integer Programming Library". In: *Mathematical Programming Computation* 13.3 (Sept. 2021), pp. 443–490. DOI: 10.1007/s12532-020-00194-3.

[5] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärtschi, Harry Buhrman, Carleton Coffrin, Giorgio Cortiana, Vedran Dunjko, Daniel J. Egger, Bruce G. Elmegreen, Nicola Franco, Filippo Fratini, Bryce Fuller, Julien Gacon, Constantin Gonciulea, Sander Gribling, Swati Gupta, Stuart Hadfield, Raoul Heese, Gerhard Kircher, Thomas Kleinert, Thorsten Koch, Georgios Korpas, Steve Lenk, Jakub Marecek, Vanio Markov, Guglielmo Mazzola, Stefano Mensa, Naeimeh Mohseni, Giacomo Nannicini, Corey O'Meara, Elena Peña Tapia, Sebastian Pokutta, Manuel Proissl, Patrick Rebentrost, Emre Sahin, Benjamin C. B. Symons, Sabine Tornow, Víctor Valls, Stefan Woerner, Mira L. Wolf-Bauwens, Jon Yard, Sheir Yarkoni, Dirk Zechiel, Sergiy Zhuk and Christa Zoufal. "Challenges and opportunities in quantum optimization". In: *Nature Reviews Physics* 6.12 (Dec. 2024). Publisher: Nature Publishing Group, pp. 718–735. URL: https://www.nature.com/articles/s42254-024-00770-9 (visited on 22/09/2025).

[6] Sabrina Ammann, Maximilian Hess, Debora Ramacciotti, Sándor P. Fekete, Paulina L. A. Goedicke, David Gross, Andreea Lefterovici, Tobias J. Osborne, Michael Perk, Antonio Rotundo, S. E. Skelton, Sebastian Stiller and Timo de Wolff. *Realistic Runtime Analysis for Quantum Simplex Computation*. arXiv:2311.09995 [quant-ph]. Nov. 2023. URL: http://arxiv.org/abs/2311.09995 (visited on 05/05/2025).

[7] Alexey Bochkarev, Raoul Heese, Sven Jäger, Philine Schiewe and Anita Schöbel. "Quantum computing for discrete optimization: A highlight of three technologies". In: *European Journal of Operational Research* 329.3 (2026), pp. 747–766. DOI: https://doi.org/10.1016/j.ejor.2025.07.063. URL: https://www.sciencedirect.com/science/article/pii/S0377221725005880.

[8] Tameem Albash and Daniel A. Lidar. "Demonstration of a Scaling Advantage for a Quantum Annealer over Simulated Annealing". In: *Phys. Rev. X* 8.3 (July 2018). Publisher: American Physical Society, p. 031016. DOI: 10.1103/PhysRevX.8.031016.

[9] Catherine C. McGeoch and Pau Farré. "Milestones on the Quantum Utility Highway: Quantum Annealing Case Study". In: *ACM Transactions on Quantum Computing* 5.1 (Dec. 2023). Place: New York, NY, USA Publisher: Association for Computing Machinery. DOI: 10.1145/3625307.

[10] Edward Farhi, Jeffrey Goldstone and Sam Gutmann. *A Quantum Approximate Optimization Algorithm*. arXiv:1411.4028 [quant-ph]. Nov. 2014. URL: http://arxiv.org/abs/1411.4028 (visited on 22/09/2025).

[11] Matthew P. Harrigan, Kevin J. Sung, Matthew Neeley, Kevin J. Satzinger, Frank Arute, Kunal Arya, Juan Atalaya, Joseph C. Bardin, Rami Barends, Sergio Boixo, Michael Broughton, Bob B. Buckley, David A. Buell, Brian Burkett, Nicholas Bushnell, Yu Chen, Zijun Chen, Ben Chiaro, Roberto Collins, William Courtney, Sean Demura, Andrew Dunsworth, Daniel Eppens, Austin Fowler, Brooks Foxen, Craig Gidney, Marissa Giustina, Rob Graff, Steve Habegger, Alan Ho, Sabrina Hong, Trent Huang, L. B. Ioffe, Sergei V. Isakov, Evan Jeffrey, Zhang Jiang, Cody Jones, Dvir Kafri, Kostyantyn Kechedzhi, Julian Kelly, Seon Kim, Paul V. Klimov, Alexander N. Korotkov, Fedor Kostritsa, David Landhuis, Pavel Laptev, Mike Lindmark, Martin Leib, Orion Martin, John

M. Martinis, Jarrod R. McClean, Matt McEwen, Anthony Megrant, Xiao Mi, Masoud Mohseni, Wojciech Mruczkiewicz, Josh Mutus, Ofer Naaman, Charles Neill, Florian Neukart, Murphy Yuezhen Niu, Thomas E. O'Brien, Bryan O'Gorman, Eric Ostby, Andre Petukhov, Harald Putterman, Chris Quintana, Pedram Roushan, Nicholas C. Rubin, Daniel Sank, Andrea Skolik, Vadim Smelyanskiy, Doug Strain, Michael Streif, Marco Szalay, Amit Vainsencher, Theodore White, Z. Jamie Yao, Ping Yeh, Adam Zalcman, Leo Zhou, Hartmut Neven, Dave Bacon, Erik Lucero, Edward Farhi and Ryan Babbush. "Quantum approximate optimization of non-planar graph problems on a planar superconducting processor". In: *Nature Physics* 17.3 (Feb. 2021). Publisher: Springer Science and Business Media LLC, pp. 332–336. DOI: 10.1038/s41567-020-01105-y.

[12] Thorsten Koch, David E. Bernal Neira, Ying Chen, Giorgio Cortiana, Daniel J. Egger, Raoul Heese, Narendra N. Hegade, Alejandro Gomez Cadavid, Rhea Huang, Toshinari Itoko, Thomas Kleinert, Pedro Maciel Xavier, Naeimeh Mohseni, Jhon A. Montanez-Barrera, Koji Nakano, Giacomo Nannicini, Corey O'Meara, Justin Pauckert, Manuel Proissl, Anurag Ramesh, Maximilian Schicker, Noriaki Shimada, Mitsuharu Takeori, Victor Valls, David Van Bulck, Stefan Woerner and Christa Zoufal. "Quantum Optimization Benchmarking Library - The Intractable Decathlon". In: *Nature Computational Science* 6.6 (June 2026). arXiv:2504.03832 [quant-ph], pp. 653–671. DOI: 10.1038/s43588-026-00991-1. URL: http://arxiv.org/abs/2504.03832 (visited on 01/10/2026).

[13] Wolfgang Sakuler, Johannes M. Oberreuter, Riccardo Aiolfi, Luca Asproni, Branislav Roman and Jürgen Schiefer. *A real world test of Portfolio Optimization with Quantum Annealing*. arXiv:2303.12601 [quant-ph]. Mar. 2023. URL: http://arxiv.org/abs/2303.12601 (visited on 11/06/2024).

[14] Chen-Yu Liu and Hsi-Sheng Goan. *Hybrid Gate-Based and Annealing Quantum Computing for Large-Size Ising Problems*. arXiv:2208.03283 [quant-ph]. Aug. 2022. URL: http://arxiv.org/abs/2208.03283 (visited on 22/09/2025).

[15] Atithi Acharya, Romina Yalovetzky, Pierre Minssen, Shouvanik Chakrabarti, Ruslan Shaydulin, Rudy Raymond, Yue Sun, Dylan Herman, Ruben S. Andrist, Grant Salton, Martin J. A. Schuetz, Helmut G. Katzgraber and Marco Pistoia. "Decomposition Pipeline for Large-Scale Portfolio Optimization with Applications to Near-Term Quantum Computing". In: *Physical Review Research* 7.2 (May 2025). arXiv:2409.10301 [math.OC], p. 023142. DOI: 10.1103/PhysRevResearch.7.023142. URL: http://arxiv.org/abs/2409.10301 (visited on 01/10/2026).

[16] Frank Phillipson and Harshil Singh Bhatia. "Portfolio Optimisation Using the D-Wave Quantum Annealer". In: *Computational Science – ICCS 2021*. Ed. by Maciej Paszynski, Dieter Kranzlmüller, Valeria V. Krzhizhanovskaya, Jack J. Dongarra and Peter M. A. Sloot. Cham: Springer International Publishing, 2021, pp. 45–59. DOI: 10.1007/978-3-030-77980-1_4.

[17] Samuel Palmer, Serkan Sahin, Rodrigo Hernandez, Samuel Mugel and Roman Orus. *Quantum Portfolio Optimization with Investment Bands and Target Volatility*. arXiv:2106.06735 [q-fin]. Aug. 2021. URL: http://arxiv.org/abs/2106.06735 (visited on 30/12/2024).

[18] Davide Venturelli and Alexei Kondratyev. "Reverse Quantum Annealing Approach to Portfolio Optimization Problems". In: *Quantum Machine Intelligence* 1.1-2 (May 2019). arXiv:1810.08584 [quant-ph], pp. 17–30. DOI: 10.1007/s42484-019-00001-w. URL: http://arxiv.org/abs/1810.08584 (visited on 01/10/2026).

[19] Sebastian Brandhofer, Daniel Braun, Vanessa Dehn, Gerhard Hellstern, Matthias Hüls, Yanjun Ji, Ilia Polian, Amandeep Singh Bhatia and Thomas Wellens. "Benchmarking the performance of portfolio optimization with QAOA". In: *Quantum Information Processing* 22.1 (Dec. 2022). arXiv:2207.10555 [quant-ph], p. 25. DOI: 10.1007/s11128-022-03766-5. URL: http://arxiv.org/abs/2207.10555 (visited on 30/12/2024).

[20] Mark Hodson, Brendan Ruck, Hugh Ong, David Garvin and Stefan Dulman. *Portfolio rebalancing experiments using the Quantum Alternating Operator Ansatz*. arXiv:1911.05296 [quant-ph]. Nov. 2019. URL: http://arxiv.org/abs/1911.05296 (visited on 21/08/2025).

[21] Zhijie Tang, Alex Lu Dou and Arit Kumar Bishwas. *Comparative analysis of diverse methodologies for portfolio optimization leveraging quantum annealing techniques*. arXiv:2403.02599 [quant-ph]. July 2024. URL: http://arxiv.org/abs/2403.02599 (visited on 22/09/2025).

[22] Giuseppe Buonaiuto, Francesco Gargiulo, Giuseppe De Pietro, Massimo Esposito and Marco Pota. "Best practices for portfolio optimization by quantum computing, experimented on real quantum devices". In: *Scientific Reports* 13.1 (Nov. 2023), p. 19434. URL: https://www.nature.com/articles/s41598-023-45392-w (visited on 04/08/2025).

[23] Francesco Cesarone, Andrea Scozzari and Fabio Tardella. "Efficient Algorithms for mean-variance portfolio optimization with Hard Real -World Constraints". In: *Giornale dell'Istituto Italiano degli Attuari* (2009), pp. 37–56.

[24] Álvaro Rubio-García, Juan José García-Ripoll and Diego Porras. *Portfolio optimization with discrete simulated annealing*. arXiv:2210.00807 [cond-mat]. Oct. 2022. URL: http://arxiv.org/abs/2210.00807 (visited on 30/12/2024).

[25] T.-J. Chang, N. Meade, J.E. Beasley and Y.M. Sharaiha. "Heuristics for cardinality constrained portfolio optimisation". In: *Computers & Operations Research* 27.13 (Nov. 2000), pp. 1271–1302. URL: https://linkinghub.elsevier.com/retrieve/pii/S030505489900074X (visited on 05/06/2025).

[26] Cassidy K. Buhler and Hande Y. Benson. *Efficient Solution of Portfolio Optimization Problems via Dimension Reduction and Sparsification*. arXiv:2306.12639 [q-fin]. June 2023. URL: http://arxiv.org/abs/2306.12639 (visited on 07/01/2025).

[27] Edoardo Alessandroni, Sergi Ramos-Calderer, Ingo Roth, Emiliano Traversi and Leandro Aolita. "Alleviating the quantum Big-M problem". In: *npj Quantum Information* 11.1 (July 2025), p. 125. URL: https://www.nature.com/articles/s41534-025-01067-0 (visited on 30/07/2025).

[28] Mahdi Moeini. *A Continuous Optimization Approach for the Financial Portfolio Selection under Discrete Asset Choice Constraints*. arXiv:1404.3286 [cs]. Apr. 2014. URL: http://arxiv.org/abs/1404.3286 (visited on 13/06/2025).

[29] Francesco Cesarone, Andrea Scozzari and Fabio Tardella. "Portfolio selection problems in practice: a comparison between linear and quadratic optimization models". In: *Computational Management Science* 12.3 (July 2015). arXiv:1105.3594 [q-fin], pp. 345–370. DOI: 10.1007/s10287-014-0210-1. URL: http://arxiv.org/abs/1105.3594 (visited on 13/06/2025).

[30] Sarat Moka, Matias Quiroz, Vali Asimit and Samuel Muller. *A Scalable Gradient-Based Optimization Framework for Sparse Minimum-Variance Portfolio Selection*. arXiv:2505.10099 [stat]. May 2025. URL: http://arxiv.org/abs/2505.10099 (visited on 13/06/2025).

[31] *IBM ILOG CPLEX Optimization Studio*. Dec. 2022. URL: https://www.ibm.com/docs/en/icos/22.1.1?topic=cplex-meet (visited on 21/08/2025).

[32] Sebastian Schlütter, Tomislav Maras, Alexander Dotterweich and Nico Piatkowski. *Hot-Starting Quantum Portfolio Optimization*. 2025. arXiv: 2510.11153 [quant-ph]. URL: https://arxiv.org/abs/2510.11153.

[33] Vicente P. Soloviev, Antonio Márquez Romero, Josh Kirsopp and Michal Krompiec. *Scaling Portfolio Diversification with Quantum Circuit Cutting Techniques*. arXiv:2506.08947 [quant-ph]. June 2025. URL: http://arxiv.org/abs/2506.08947 (visited on 13/06/2025).

[34] Zi Xuan Loke, Say Leng Goh, Graham Kendall, Salwani Abdullah and Nasser R. Sabar. "Portfolio Optimization Problem: A Taxonomic Review of Solution Methodologies". In: *IEEE Access* 11 (2023), pp. 33100–33120. URL: https://ieeexplore.ieee.org/document/10087257/ (visited on 07/01/2025).

[35] Harry Markowitz. "Portfolio Selection". In: *The Journal of Finance* 7.1 (1952), pp. 77–91. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1952.tb01525.x (visited on 05/03/2025).

[36] Stephen P. Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge, UK ; New York: Cambridge University Press, 2004.

[37] Margaret Wright. "The interior-point revolution in optimization: History, recent developments, and lasting consequences". In: *Bulletin of the American Mathematical Society* 42.1 (Sept. 2004), pp. 39–56. URL: https://www.ams.org/bull/2005-42-01/S0273-0979-04-01040-7/ (visited on 22/09/2025).

[38] *Stock Screener NASDAQ*. URL: https://www.nasdaq.com/market-activity/stocks/screener (visited on 26/05/2026).

[39] *Yahoo Finance - Stock Market Live, Quotes, Business & Finance News*. URL: https://finance.yahoo.com/ (visited on 13/06/2025).

[40] *yfinance: Python Package to download market data from Yahoo! Finance API*. Sept. 2025. URL: https://pypi.org/project/yfinance/0.2.66/ (visited on 01/10/2026).

[41] Suresh Bolusani, Mathieu Besancon, Ksenia Bestuzheva, Antonia Chmiela, Joao Dionısio, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Mohammed Ghannam, Ambros Gleixner, Christoph Graczyk, Katrin Halbig, Ivo Hedtke, Alexander Hoen and Christopher Hojny. *The SCIP Optimization Suite 9.0*. arXiv:2402.17702 [math.OC]. 2024. URL: https://arxiv.org/abs/2402.17702v2 (visited on 26/05/2026).

[42] *PySCIPOpt (Python Package for SCIP)*. Aug. 2025. URL: https://pypi.org/project/PySCIPOpt/5.6.0/ (visited on 01/10/2026).

[43] *The Leader in Decision Intelligence Technology - Gurobi Optimization*. URL: https://www.gurobi.com/ (visited on 04/03/2025).

[44] Philipp Hauke, Helmut G. Katzgraber, Wolfgang Lechner, Hidetoshi Nishimori and William D. Oliver. "Perspectives of quantum annealing: Methods and implementations". In: *Reports on Progress in Physics* 83.5 (May 2020). arXiv:1903.06559 [quant-ph], p. 054401. DOI: 10.1088/1361-6633/ab85b8. URL: http://arxiv.org/abs/1903.06559 (visited on 30/05/2025).

[45] Tameem Albash and Daniel Lidar. "Adiabatic quantum computation". In: *Reviews of Modern Physics* 90.1 (2018). DOI: 10.1103/RevModPhys.90.015002.

[46] Albert Messiah. *Quantum Mechanics*. Google-Books-ID: VR93vUk8d_8C. Elsevier, 1961.

[47] *The Advantage™ Quantum Computer | D-Wave*. URL: https://www.dwavequantum.com/solutions-and-products/systems/ (visited on 22/09/2025).

[48] Vicky Choi. "Minor-Embedding in Adiabatic Quantum Computation: I. The Parameter Setting Problem". In: *Quantum Information Processing, 7* (Apr. 2008). arXiv:0804.4884 [quant-ph], pp193–209. DOI: 10.48550/arXiv.0804.4884. URL: http://arxiv.org/abs/0804.4884 (visited on 01/10/2026).

[49] Vicky Choi. "Minor-embedding in adiabatic quantum computation: II. Minor-universal graph design". In: *Quantum Information Processing* 10.3 (June 2011). arXiv:1001.3116 [quant-ph], pp. 343–353. URL: http://arxiv.org/abs/1001.3116 (visited on 22/09/2025).

[50] *Samplers — dwave-system 0.9.12 documentation*. URL: https://dwave-systemdocs.readthedocs.io/en/latest/reference/samplers.html#dwavecliquesampler (visited on 24/06/2025).

[51] *dwave-system (Python Package for Dwave)*. Sept. 2025. URL: https://pypi.org/project/dwave-system/1.33.0/ (visited on 01/10/2026).

[52] Jun Cai, William G. Macready and Aidan Roy. *A practical heuristic for finding graph minors*. arXiv:1406.2741 [quant-ph]. June 2014. URL: http://arxiv.org/abs/1406.2741 (visited on 22/09/2025).

[53] Mostafa Khezri, Xi Dai, Rui Yang, Tameem Albash, Adrian Lupascu and Daniel A. Lidar. "Customized Quantum Annealing Schedules". In: *Physical Review Applied* 17.4 (Apr. 2022). DOI: 10.1103/physrevapplied.17.044005. URL: http://dx.doi.org/10.1103/PhysRevApplied.17.044005.

[54] Asier Ozaeta, Wim Van Dam and Peter L McMahon. "Expectation values from the single-layer quantum approximate optimization algorithm on Ising problems". In: *Quantum Science and Technology* 7.4 (Oct. 2022), p. 045036. URL: https://iopscience.iop.org/article/10.1088/2058-9565/ac9013 (visited on 11/06/2024).

[55] J. A. Montanez-Barrera and Kristel Michielsen. "Towards a Linear-Ramp QAOA protocol: Evidence of a scaling advantage in solving some combinatorial optimization problems". In: *npj Quantum Information* 11.1 (Aug. 2025). arXiv:2405.09169 [quant-ph], p. 131. DOI: 10.1038/s41534-025-01082-1. URL: http://arxiv.org/abs/2405.09169 (visited on 01/10/2026).

[56] *minimize(method='COBYLA') — SciPy v1.15.3 Manual*. URL: https://docs.scipy.org/doc/scipy/reference/optimize.minimize-cobyla.html (visited on 30/05/2025).

[57] *scipy (open-source software for mathematics, science, and engineering)#*. Sept. 2025. URL: https://pypi.org/project/scipy/1.16.2/ (visited on 01/10/2026).

[58] *AerSimulator - Qiskit Aer 0.16.1*. URL: https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.AerSimulator.html (visited on 30/05/2025).

[59] *qiskit-aer (Python Package for quantum circuit simulation)*. Sept. 2025. URL: https://pypi.org/project/qiskit-aer/0.17.2/ (visited on 01/10/2026).

[60] M. AbuGhanem. "IBM Quantum Computers: Evolution, Performance, and Future Directions". In: *The Journal of Supercomputing* 81.5 (Apr. 2025). arXiv:2410.00916 [quant-ph], p. 687. DOI: 10.1007/s11227-025-07047-7. URL: http://arxiv.org/abs/2410.00916 (visited on 01/10/2026).

[61] *generate_preset_pass_manager (v2.0)*. URL: https://quantum.cloud.ibm.com/docs/en/api/qiskit/2.0/quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.transpiler.generate_preset_pass_manager (visited on 21/09/2026).

[62] Daniel J. Egger, Jakub Mareček and Stefan Woerner. "Warm-starting quantum optimization". In: *Quantum* 5 (2021), p. 479. DOI: 10.22331/q-2021-06-17-479. URL: http://dx.doi.org/10.22331/q-2021-06-17-479.

[63] J. V. S. Scursulim, Gabriel M. Langeloh, Victor L. Beltran and Samuraí Brito. "Multiclass portfolio optimization via variational quantum Eigensolver with Dicke state ansatz". In: *Scientific Reports* 16.1 (Feb. 2026). DOI: 10.1038/s41598-026-36333-4. URL: http://dx.doi.org/10.1038/s41598-026-36333-4.

[64] *dwavesystems/dwave-greedy*. original-date: 2019-08-02T10:10:41Z. Sept. 2024. URL: https://github.com/dwavesystems/dwave-greedy (visited on 30/05/2025).

[65] *dwave-greedy (Python Package for Dwave Greedy)*. Nov. 2022. URL: https://pypi.org/project/dwave-greedy/0.3.0/ (visited on 01/10/2026).

[66] S. Kirkpatrick, C. D. Gelatt and M. P. Vecchi. "Optimization by Simulated Annealing". In: *Science*. New Series 220.4598 (1983), pp. 671–680. DOI: 10.1126/science.220.4598.671. URL: http://www.jstor.org/stable/1690046.

[67] *dwave-samplers — Python documentation*. URL: https://docs.dwavequantum.com/en/latest/ocean/api_ref_samplers/index.html (visited on 30/05/2025).

[68] *dwave-samplers (Python Package for Dwave samplers)*. Oct. 2025. URL: https://pypi.org/project/dwave-samplers/1.7.0/ (visited on 01/10/2026).

[69] *dwavesystems/dwave-tabu*. original-date: 2018-08-28T02:14:30Z. Sept. 2024. URL: https://github.com/dwavesystems/dwave-tabu (visited on 30/05/2025).

[70] *dwave-tabu (Python Package for Dwave Tabu)*. Nov. 2022. URL: https://pypi.org/project/dwave-tabu/0.5.0/ (visited on 01/10/2026).

[71] Arturo Acuaviva, David Aguirre, Rubén Peña and Mikel Sanz. "Benchmarking quantum computers: towards a standard performance evaluation approach". In: *Quantum Science and Technology* 11.2 (Feb. 2026), p. 025004. DOI: 10.1088/2058-9565/ae4269. URL: https://doi.org/10.1088/2058-9565/ae4269 (visited on 01/10/2026).

[72] *stopfereric/portfolio_opt_benchmark*. URL: https://github.com/stopfereric/portfolio_opt_benchmark (visited on 22/09/2025).

[73] Robert C. Merton. "An Analytic Derivation of the Efficient Portfolio Frontier". In: *The Journal of Financial and Quantitative Analysis* 7.4 (1972), pp. 1851–1872. URL: http://www.jstor.org/stable/2329621 (visited on 03/07/2026).